

Programmeringstermer

Lite hjälp i C# kodningen

V 1.04

Marcus Marcelo Medina

Skapad 2009-11-03

Senast ändrad 2009-12-27

Det här dokumentet ger en enkel beskrivning (med kodexempel) på termer som används inom programmering och C#

INNEHÅLL

Inledning	6
Förändringar.....	7
Class	9
Class Benämningar	9
Constructor	9
Destructor	10
classmetod	10
Classvariabler	10
Fält.....	10
Instans	10
Instansiering.....	10
Standardmetoder	10
Equals().....	11
GetType	11
Gethashcode().....	11
ToString().....	12
Debug med ToString()	12
Metod.....	13
Metodhuvud.....	13
Metodanrop	13
metodnamn.....	13
Overload	13
Object.....	14
objektmetod.....	14
Variabler.....	14
Variablers existensarea	14
scope	14
Variabelnamn	15
Deklarering.....	15
Tilldelning.....	15
Initiering	15
Public.....	15
Private	15
Static.....	16
ref.....	16
out.....	16
param	16
parametrar	16
argument.....	16
Const	17
Enum	17
Struct	19
Indexerare	20
Arrays	20
Position vs antal	21
Tvådimensionella arrayer.....	21
Flerdimensionella arrayer	22

Abstraktion.....	22
List<T>	22
Abstrakta classer	23
Interface	24
IComparable.....	24
IComparable<T>.....	25
IEnumerable	25
IEnumerator	25
IEnumerator<T>	25
Generiska klasser	26
Algoritm	26
Exceptions	26
Try Catch	26
Finally	27
Lite att tänka på	27
....men	27
Loop	27
Oändlig loop	28
For loop	28
Foreach	28
While loop	29
Do loop.....	30
Nästlade loopar	30
Subrutin.....	31
OOP – Objektorienterad programmering	32
Arv	32
Base.....	32
This	32
is.....	33
Virtual.....	33
Override	33
New	33
Sealed.....	34
Get.....	34
Set	34
Egenskaper (Get/Set)	34
Skrivskyddade egenskaper	35
"Fejkade" egenskaper	35
value.....	35
Polymorfism	35
Namngivningskonventioner	36
Ungersk notation.....	36
Camel notation.....	36
Pascal notation.....	36
Övrig information.....	37
Kodlägen.....	37
Designtime	37
Runtime.....	37
Typer i minnet	38

Value type (ByVal)	38
reference type (ByRef)	38
Typsäkring	39
Casting.....	39
Garbage collector	40
Pseudokod.....	40
Kodexempel	41
Selektion.....	41
Val	41
Flödesschema.....	42
Kodexempel	42
Selektion.....	43
Loop	43
Val	43
Kodexempel	44
Constructor	44
Destructor	44
Destructor trigger.....	44
Instanciering.....	44
Classmetod.....	45
Variabler	45
Statisk fält.....	45
Metoder	46
Overload.....	46
Metodanrop	46
Variabelnamn	47
Instansmetoder och instansvariabler.....	47
Const	48
Const i praktiken, exempel 1.....	48
Const i praktiken, exempel 2.....	48
ref.....	49
out.....	49
Byval och ByRef	49
GetType	50
Jämförelse av typ med GetType.....	50
Jämförelse av typ med Is.....	50
Hashvärden	50
Hashvärden och classer.....	51
Hashcode.....	52
Algoritm.....	53
Hund-Class.....	53
Polymorfism	54
Base och this	54
Sealed	55
Abstrakt class	55
interface	55
Icomparable	56
Icomparable – enklare version.....	57
Icomparable <T>.....	57

IComparable<T> – enklare version	58
IEnumerator	58
Generisk class	58
Get/Set metoder	59
Egenskaper	59
En Annorlunda egenskap	60
Loop-exempel.....	60
Oändliga loopar.....	60
For-sats.....	61
While-sats	61
Do-sats	61
Nästlade loopar	62
Luffarschack	62
Klocka	62
Arrayer	63
Handellista	63
Arraypositioner	64
Internet länkar	65

INLEDNING

Här följer en rad med termer och korta beskrivningar av dem. I boken som ni rekommenderats att köpa till kursen finns alla dessa termer förklarade. Alla termer är inte förklarade, på grund av tidsbrist, glömska eller för att AP09 inte kommit till den punkten än att de behöver kunna det 😊

Fler termer dyker möjligen upp i en nyare version. Hoppas det blir till hjälp för er blivande programmerare!

Förslag och kommentarer kan skickas till hello@marcusmedina.pro

Hälsningar

Marcus Marcelo Medina

<https://marcusjobb.github.io>

FÖRÄNDRINGAR

Här följer en lista på de senaste förändringarna sen tidigare versioner.

(Den kommer att kortas ner sen så att den inte tar mer än en sida, och då visar bara 2 eller 3 versioner bakåt)

Version	Tillägg/Förändring	Kommentar
1.03	Nya länkar i länklistan	Roligt roligt
1.03	Generiska classer och exempel	Ordförklaring och exempel
1.03	Exceptions	Ordförklaring
1.03	Scope	Ordförklaring
1.03	Variablers existensarea	Ordförklaring
1.03	Indexer	Ordförklaring
1.03	Polymorfism	Ordförklaring
1.03	Reference type	Ordförklaring
1.03	Value type och exempel	Ordförklaring
1.03	Icomparable och IComparable<>	Ordförklaring
1.03	Ienumerable	Ordförklaring
1.03	IEnumerator / IEnumerator<>	Ordförklaring
1.03	List<T>	Ordförklaring
1.03	Const	Ordförklaring
1.03	Enum	Ordförklaring
1.03	struct	Ordförklaring
1.03	Base	Ordförklaring
1.03	This	Ordförklaring
1.03	is	Ordförklaring
1.03	Debug med toString()	Ordförklaring
1.03	Standardmetod: ToString()	Metodbeskrivning
1.03	Standardmetod: GetHashCode() och exempel	Metodbeskrivning och exempel
1.03	Standardmetod: GetType() och exempel	Metodbeskrivning
1.03	Standardmetod: Equals()	Metodbeskrivning
1.03	Casting	Ordförklaring
1.03	Typsäkring	Ordförklaring
1.03	Foreach	Ordförklaring
1.03	Kodlägen	Ordförklaring
1.03	Runtime	Ordförklaring
1.03	Design-time	Ordförklaring
1.03	Interface och exempel	Ordförklaring
1.03	Abstract class och exempel	Ordförklaring
1.03	Abstraktion	Teoretisk förklaring
1.03	Fält	Tillägg till ordförklaringen
1.03	Virtual	Ordförklaring
1.03	Override	Ordförklaring
1.03	New	Ordförklaring
1.03	Overload och exempel	Ordförklaring och kodexempel
1.03	Seal	Ordförklaring
1.03	Array Positon vs Antal, exempel	Förklaring och kodexempel
1.03	Class benämningar	Uttrycken Superclass och subclass förklaras.
1.03	Pascal notation	Tillägg till ordförklaringen, kommentar om kemi
1.02	Loop	Ordförklaring
1.02	Nästlade loopar, Luffarschack och klocka	Ordförklaring och exempel
1.02	Oändlig loop och exempel	Ordförklaring och exempel
1.02	Arrays, exempel och Handellista	Ordförklaring och exempel
1.02	While-sats och exempel	Ordförklaring och exempel
1.02	For-sats och exempel	Ordförklaring och exempel
1.02	Do-sats och exempel	Ordförklaring och exempel
1.02	Inledning	Uppdaterad
1.02	Förändringar sen förra versionen	Ny avdelning
1.01	Flödesschema Val	Korrigerad
1.01	Flödesschema Selektion	Korrigerad
1.01	Variabelnamn	Korrigerade felstavning
1.01	Algoritm och exempel	Ordförklaring och exempel
1.01	Subrutin	Ordförklaring

1.01	OOP	Ordförklaring
1.01	Arv och Hundexempel	Ordförklaring och exempel
1.01	Get / Set och exempel	Ordförklaring och exempel
1.01	Skrivskyddade egenskaper	Ordförklaring
1.01	"Fejkade egenskaper"	Ordförklaring
1.01	Value variabeln	Ordförklaring
1.01	En annorlunda egenskap	Exempel
1.01	Egenskaper	Exempel

CLASS

En class är en slags behållare av programkod. I den kan du lagra **Metoder**, **Variabler**, Strukturer, **egenskaper** och annat skoj. Du kan även lagra andra classer i classen. Classen definieras med nyckelordet Class och dess innehåll läggs inom måsvingar { }. Classer har en **constructor** och en **destructor**.

CLASS BENÄMNINGAR

En klass kan ha olika benämningar beroende på hur det används.

Superclass eller Base-class: Klassen som andra classer ärver ifrån.

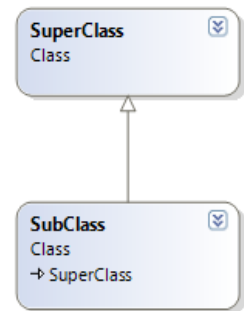
Subclass: Klassen som ärver från en superclass.

Uttrycket "att subclassa" är samma sak som att säga att man ärva från en class.

```
class SuperClass{ }  
class SubClass : SuperClass { }
```

Detta hindrar förstås inte att en annan class ärver subclassen, då blir den en subclass och subclassen betraktas som superclass.

*Alla klasser är egentligen subclasser till **Object** classen ☺*



CONSTRUCTOR

En startmetod för Classer, det är den första som anropas när classen **instancieras**. Den heter likadant som Classen och kan ta emot parametrar, däremot så returnerar den ingenting.

Även om constructorn är anpassningsbar så är den också ovillkorlig för användaren. Om man har en enbart en constructor som tar en sträng så kan man inte skapa en instans av Classen utan att ange en sträng till constructorn.

Constructorn kan också vara privat, då använder man vanligen en statisk metod för att anropa det, man gör så ibland för att försäkra sig om att saker och ting sker på rätt sätt. Om du exempelvis har en class som i constructorn ska ta emot en annan class, så vill du gärna försäkra dig om att den classen skapats på rätt sätt. Constructorn används oftast för att tilldela standardvärden till classens variabler.

Constructorn används vanligast till att förse classen med standardvärden.

Kodexempel: **Constructor**

DESTRUCTOR

Constructorn är det första som körs när en class instancieras, destructorn är raka motsatsen. Den körs när classen förstörs. Vanligen körs den när Windows försöker rensa upp minnet eller när programmet kört till sitt slut.

Kodexempel: [Destructor](#) och [Destructor trigger](#)

CLASSMETOD

En statisk metod som finns i en class. Det innebär att man inte behöver instanciera classen för att köra metoden. Exempel på sådana är `Console.WriteLine()`; (man instancierar inte Console classen, eller hur?)

Se [Classmetod](#)

CLASSVARIABLER

Se [fält](#)

FÄLT

En variabel som deklareras i **classen** (kallas field av Visual Studios classdiagram).

Kan även beteckna en **array**.

Se [variabler](#)

INSTANS

En instans är ett object eller class som lagrats i en variabel. Genom instancieringen kan vi kolla på classens instansvariabler och instansmetoder.

Se exempel [Instansmetoder och instansvariabler](#)

INSTANSIERING

Instanciering av ett objekt är handlingen man gör när man skapar den i en variabel.

Kodexempel: [Instanciering](#)

STANDARDMETODER

Alla klasser har vissa standardmetoder, dessa läggs till automatiskt och kan inte tas bort. Däremot kan man **overrida** dem. De tillhör nämligen **"Object"** classen som är superclassen till alla classer.

EQUALS()

Equals är ett sätt att jämföra objekt på. Koden ser ut såhär

```
public override bool Equals(object obj)
{
    return base.Equals(obj);
}
```

Denna metod används för att jämföra två objekt. Det enda denna metod säger är om ett objekt är likadan som ett annat objekt. Genom att **overrida** den kan man få jämförelsen att jämföra på det sätt man vill. Exempelvis om du har en class som fungerar som personregister, då kan du jämföra personerna baserat på personnummer, vikt på frukten, eller eankod...

```
MinClass Other=(MinClass)obj;//Casta obj till att bli av sorten MinClass
return personNummer==other.personNummer
```

Denna metod används när man skriver exempelvis

```
if (x == 5) { }
```

Man kan göra likadant med sina egna klasser, exempelvis

```
if (minClass[0]==minClass[1]) { }
```

Om man kör den här jämförelsen utan att implementera Equals() metoden kommer jämförelsen att baseras på classens Hash värde (se [GetHashCode](#)) vilket alltid kommer att returnera fel om man skapat olika instanser av en class, även om man gett den samma värden. Ett personregister skulle bli helt fel om man jämförde på det här sättet, alltså måste man göra jämförelsen på egen hand som exemplet ovan.

GETTYPE

Denna metod returnerar typen på en class, ifall man är osäker på typen av ett visst objekt. Detta kan ske när man har flera olika objekt i en lista som inte är typad. För mer info se [Typsäkring](#).

Se kodexempel [GetType](#)

GETHASHCODE()

Hashkoden är en kod som genereras automatiskt för alla typer och klasser som instansieras. Den används för att skilja på dem. Det finns även Hashlistor som gör att man försäkras sig om att ha unika värden i listan. Men i detta fallet handlar det om att få reda på hashvärdet till vår egenhändigt skapade klass.

Ett objekt som inte implementerar GetHashCode får ett automatiskt värde baserat på en algoritm i .net. Vill vi kontrollera detta måste vi skapa en egen Hashgenerator.

Vanliga talvärden har samma hashvärden som det värdet de har.

```
int x=5;
Console.WriteLine(x.GetHashCode()); // Skriver ut 5
```

Det finns en speciell algoritm för strängar som ger samma resultat oavsett hur många instanser av strängarna som man instancierar med samma värden. Så är det för alla andra typer som finns inbyggda i .net.

Om vi exempelvis gör en personclass som implementerar sin egen GetHashCode baserad på personnummer, kan vi då enkelt kontrollera om samma person matats in flera gånger.

Hashvärdet är bra till mycket!

Se kodexempel Hashvärden

TOSTRING()

ToString är en standardmetod man använder för att hämta sträng information från en class. Egentligen har man väldigt mycket nytta av den här metoden om man bara tar sig tid att lära känna den.

Denna metoden är bekant från när vi omvandlat int eller long till strängvärden. Man kan även använda ToString för att läsa av namnet på en enum!

Metoden finns inbyggt i alla classer men de flesta returnerar en sträng med namnet på typen av classen. Det är självklart vad vi inte vill ha, men det kan fixas ganska lätt.

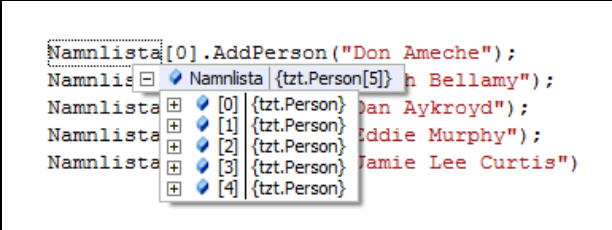
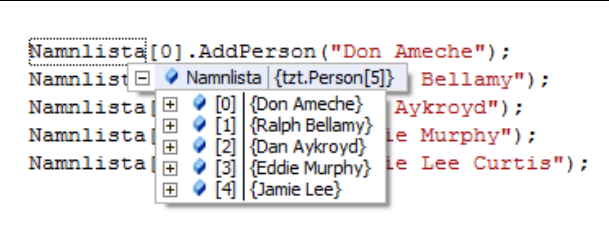
Man kan **övertida** ToString() och göra så att den returnera vad vi vill. Exempelvis om vi har en class för att handskas med personer, skulle vi kunna övertida ToString på detta sätt.

```
public override string ToString()
{
    return namn + " " + efternamn;
}
```

Detta ger många fördelar, man får snabb tillgång till hela namnet. Om man skickar in classen till en Lista eller ListBox i ett Windows Forms formulär så visas värdet från ToString(), alltså behöver man inte göra mer än så.

DEBUG MED TOSTRING()

Det finns även en mer dold fördel som många inte känner till. När man debuggar exempelvis array med classer som man själv skapat, så kan man använda ToString() för att få fram debugvärdena man vill se. Det hjälper inte användarna men det gör programmerandet betydligt enklare!

Debug innan ToString() Override	Debug efter ToString() Override
	

METOD

En metod är en kodsnuitt som utför en del handling för att sedan återvända till den plats där den anropades ifrån. Den kan ta emot och returnera värden, enligt det metodhuvud som angivits.

Exempel på metoder: När man instancierat en class så anropad **constructor** metoden. När programmet startar anropas **main()** metoden.

Se kodexempel [Metoder](#)

METHODHUVUD

Metodhuvudet är första raden i metoden, där man deklarerar vad den ska ha för åtkomstgrad, variabel typ, namn och in-parameter.

Först väljer man åtkomstgrad	<code>[private public osv...]</code>
Sedan anger man om den är statisk	<code>[static]</code>
Därefter anger man variabeltyp den ska returnera	<code>[void int string osv...]</code>
Sedan ger man namn på metoden	<code>NamnPåMetoden()</code>
och slutligen inparametrar	<code>(typ Variabelnamn)</code>

METODANROP

Metodanrop är när man kallar på en metod för att den ska utföra en viss handling. Oftast gör man detta för att man vill ha fram viss information. Exempelvis om man har en beräkning som ska utföras flera gånger med olika utgångspunkter.

Se kodexempel [Metodanrop](#)

METHODNAMN

Det namn man ger en metod `StällEnFråga();` eller `GissaEttTal();` det bästa är att välja ett beskrivande namn för metoden, metoder som `MinMetod();` eller `DoSomething();` fungerar fint, men gör koden svårare att läsa, då man inte förstår vad metoden ska göra.

OVERLOAD

Overload är när man gör flera versioner av en metod, fast med olika parametrar. Vanligen gör man det med **constructorn** i en **class** när man har flera stycken i samma class, men det fungerar fint med vanliga metoder också. Man får inte ha overrides som tar emot exakt samma sorts parametrar i exakt samma ordning.

Se kodexempel [Overload](#)

OBJECT

I C# är allting objekt. Object classen är en standardtyp inbyggd i C# där alla andra typer har sin grund. I den finns standardmetoder som kan återfinnas i de flesta typer och classer.

OBJEKTMETOD

Eh.. uh... en metod som finns inne i ett objekt (exempelvis metoder i en **class**). 😊

VARIABLER

Variabler kallas som sådana för att deras värde och innehåll varierar från gång till gång, det beror helt enkelt på vilken typ och vilket innehåll man ger dem. Vanligen deklarerar man variabeln först, och sedan tilldelar man ett värde.

VARIABLERS EXISTENSAREA

Variabler kan finnas i olika områden i ditt program.

- Globala variabler är de som finns tillgängliga för hela classen.
- Lokala variabler finns inom en metod eller en kodsnuitt (for / while etc)
Exempelvis
 - En for sats deklarerar en lokal variabel som dör efter att forsatsen avslutat
 - En while sats kan innehålla variabler som gäller så länge while satsen körs

SCOPE

Man brukar säga att datan finns inom ett scope, alltså en vidd. Viddens utsträckning bestäms av måsvingarna.

Man kan säga att variablerna lever inom måsvingarna, allt som finns inom dessa måsvingar kan nå variablerna.

Exempelvis

```
class minClass
{
    int a=1;
    public void VisaA()
    {
        int b = 2;
        Console.WriteLine("a+b={0}", a + b);

        {
            int c = 4;
            Console.WriteLine("a+b+c={0}", a + b+c);
        }
    }
}
```

Allt inom classen kan nå variabeln **a**, men variabeln **b** är bara tillgänglig för metoden VisaA, dock är den tillgänglig för allt inom den metoden. Variabeln **c** däremot är deklarerad inom en uppsättning måsvingar (en scope), även om det inte är en for, if eller while-sats så kan man lägga kod inom måsvingar för att skärma av från resten. I detta fall kommer **c** att finnas enbart så länge koden körs innanför de måsvingarna. Men **b** kommer att finnas så länge metoden körs. Variabeln **a** kommer att finnas så länge klassen är instansierad.

VARIABELNAMN

Namnet på variabeln, detta namn anges för att det ska vara lättare för programmeraren att förstå vad den ska användas till. Det bästa är att använda beskrivande (men inte för långa) variabelnamn.

Se kodexempel [Variabelnamn](#)

DEKLARERING

När man deklarerar variabeln så ger man den ett namn och väljer vilken typ av data den ska innehålla.

Kodexempel: [Variabler](#)

TILLDELNING

När man ger en variabel ett värde, så kallas handlingen tilldelning. Detta sker antingen vid deklarationen av variabeln eller vid något senare tillfälle. Om inget deklarerats blir variabeln antingen Null, String.Empty, 0 eller något annat standardvärde.

Kodexempel: [Variabler](#)

INITIERING

Första tillfället då man ger en variabel ett värde.

PUBLIC

En publik variabel är en sådan som kan utan hinder nås från andra classer som instancierat den aktuella classen.

PRIVATE

En privat variabel är en sådan som enbart finns tillgänglig för den aktuella classen. Även om classen instancierats så kommer variabeln att förbli dold. Man använder detta för att skydda information och för att slippa utsätta användaren för variabler som man gjort specifikt för den egna classen.

STATIC

En statisk variabel finns i classen även om den inte har blivit instancierad. Man kan nå den direkt genom att skriva classnamnet och välja variabelnamnet. Denna variabel finns tillgänglig även för instansmetoderna. Även hela classer kan göras statiska.

Se kodexempel [Statisk fält](#)

REF

I vanliga fall skickas variabler som kopior till **metoder** vid **metodanropen**, men om man vill att metoden ska kunna påverka och förändra den variabeln man skickar in, då ska man lägga till **ref** i metodanropet. Detta används inte så mycket då man ofta vill undvika att ha andra som leker med ens variabler. För att ref ska fungera måste variabeln vara initierad (ha ett värde tilldelat).

Detta sker automatiskt när man skickar in en class som inparameter. Classer skickas alltid som ref.

Se kodexempel [ref](#)

OUT

Out används för att visa att metoden returnerar ett värde i den variabeln man skickat in, på liknande sätt som ref, men skillnaden är att out inte kräver att variabeln ska vara initierat. Detta alternativ används inte heller särskilt mycket, men den finns på grund av att man behöver den då och då.

Se kodexempel [out](#)

PARAM

Param är ett sätt som inte används så mycket men som ändå finns där. Oftast ser man den i mains metodhuvud. Detta ger möjligheten att skicka in parametrar som är separeras med kommatecken.

```
static void Main(string[] args)
```

Genom att titta på innehållet i `string[] args` kan man se vilka parametrar som skickades in.

PARAMETRAR

Den information man skickar in till en metod

ARGUMENT

Informationen som skickas från consolen eller från Windows till main metoden. Ibland används ordet som synonym för parametrar.

CONST

Const är en konstant variabel man anger som har ett statiskt värde. Den kan inte ändras i **Runtime** för att kompilatorn ersätter den med det värdet som man angett.

Exempelvis om vi har följande kod

```
const int MaxVärde = 10;  
int[] minaTal = new int[MaxVärde];
```

så kommer kompilatorn att läsa koden som

```
int[] minaTal = new int[10];
```

Detta gör man för att ha fasta värden som man använder igenom hela programmet, och genom att ändra på en plats i koden så kan man påverka hela programmet och slipper därmed leta och ändra överallt.

Tips: Gör det till en regel att använda Const på alla platser där du kan tänkas använda fasta tal. Det är mycket lättare att läsa kod och korrigera kod där man kan läsa vad särskilda värden gör, än att läsa en massa siffror.

Om du ska räkna antal månader är det snyggare att skriva

```
const int antalMånader = 12;
```

än att bara hänvisa till siffran 12 i programmet.

Sådant gör din kod mycket mer läsbart!

Se kodexempel **Const**

ENUM

En enum är ett enkel class-form som enbart håller reda på nummer, dessa ersätts av ord för att göra det lättare för användare. Det är ungefär som en Const. Det finns dock tillfällen då man inte vill använda Const även om man vill ha fasta värden. Exempelvis om du har värden som räknas uppåt ett steg i taget.

```
const int Januari = 1;  
const int Februari = 2;  
const int Mars = 3;  
const int April = 4;  
const int Maj = 5;  
const int Juni = 6;  
const int Juli = 7;  
const int Augusti = 8;  
const int September = 9;  
const int Oktober = 10;  
const int November = 11;  
const int December = 12;
```

Även om det blir enklare att programmera med dessa fasta värden så får du väldigt många variabler att hålla reda på. Det gör användandet av intellisence jobbigare. Alltså får man hitta på ett mellanting.

Man skulle kunna göra en **statisk class** som innehåller dessa värden. Exempelvis

```
class Månader
{
    public const int Januari = 1;
    public const int Februari = 2;
    public const int Mars = 3;
    osv...
```

Då skulle man kunna nå de statiska värdena genom att skriva `Månader.Juli` exempelvis. Nackdelen är att du får med alla standardmetoder som alla classer har. Ingen snygg lösning med andra ord. För att slippa det problemet skapar man alltså en enum.

```
enum Månader
{
    Januari=1,
    Februari,
    Mars,
    April,
    Maj,
    Juni,
    Juli,
    Augusti,
    September,
    Oktober,
    November,
    December
}
```

Man anger ett startvärde på var den ska börja räkna ifrån, sedan behöver man inte ange övriga värden, de blir automatiskt tilldelade enligt det man angett i den första. Orden som ska ersätta siffrorna separeras med comma och man anger inget semikolon i slutet.

Genom Enum får du en ren lista med bara de fasta värden du vill ha, utan extra metoder. Du anger startvärdet för den första och sedan adderas ett till värdet automatiskt. Förutom detta finns det några andra fördelar du har med enum.

Du kan **casta** int värden till enum.

```
int minFödelsemånad=6;
Månader m=(Månader)minFödelsemånad; // m får värdet Månader.Juni.
```

Du kan även casta en enum till string med metoden **ToString()**

```
Console.WriteLine(m.ToString()); // Skriver ut Juni.
```

Slutligen kan du göra olika sorts jämförelser för att kontrollera din enum.

```
if ((int)m == 6) { } //Casta enum till int och jämför
if (m==Månader.Mars) //Jämför enum med enum
if ((Månader)annanMånad==Månader.Oktober) //Casta int till Månad och jämför
```

Enum kan användas som en typ och skickas som in och ut parameter.

De fördelarna hade du inte haft med en vanlig class innehållande en serie med const.

STRUCT

En enum kan förse en med numeriska värden som ökar för var värde i listan, men tänk om vi vill ha strings eller double?

Det är då en Struct kommer in i bilden. En struct en är lite-version av en class med vissa undantag

- Man anger ingen constructor eller destructor till den
- Den ärver inte och kan inte ärvas.
- Struct är en **Value type**, till skillnad från classer som är **Reference type**.

Just den sista skillnaden är den som bestämmer om man ska använda en Struct eller en class. Om det är små mängder av data som ska lagras och föras över, exempelvis fasta värden i data struktur så är det helt OK att använda en struct. I annat fall är det smidigare med en class då informationen inte skickas runt utan enbart ett par bytes med minnesreferens skickas vidare.

Såhär deklarerar man en struct:

```
struct PersonInfo
{
    public string namn;
    public int ålder;
}
```

Och man använder den som en vanlig class.

```
PersonInfo p = new PersonInfo();
p.namn = "Frk Fräken";
p.ålder = 27;
```

Man kan skriva metoder i structen, men det är lite dumt med tanke på att den skickas som **By value** mellan metoder, ju mer kod man lägger in i structen desto mer information skickas vidare och desto mer minne används... och desto långsammare blir programmet. Men i **debugsyfte** kan det vara bra att **övertida ToString()** i structen.

```
public override string ToString()
{
    return namn + ", " + ålder.ToString();
}
```

INDEXERARE

En indexerare är en metod, eller rättare sagt en egenskap i classen som får classen att bete sig som en array.

Om du har en class med en lista av personer, och vill att användarna av classen ska kunna fråga efter en specifik person i din lista. Då kan du använda en indexerare. Koden för indexeraren kan se ut såhär:

```
public object this[int index]
{
    get { return minLista[index] } //returnerar värdet i det givna indexet
    set { minLista[index]=value } //sparar värdet i det givna indexet
}
```

Det fungerar precis som en vanlig **egenskap** med get/set. Men den anropas lite annorlunda.

```
MinLista lst=new MinLista(); //Instansiera classen
lst[0]=10; //Tilldela ett värde genom indexer egenskapen
Console.WriteLine(lst[3]); //läs ett värde genom indexer egenskapen
```

ARRAYS

En array är en matris av data (även kallad **fält**). Exempelvis kan man ha en lista av integers, på detta sätt kan man lagra alla i med samma variabelnamn.

I vanliga fall skulle man ha skrivit

```
int värde1=10;
int värde2=11;
int värde3=15;
```

Såhär ser det ut i minnet.

Variabelnamn	Värde
värde1	11
värde2	10
värde3	15

Vill man nå informationen skriver man ett variabelnamn i taget.

Med en array skriver man istället

```
int[] värde={10,11,15};
```

Såhär ser det ut i minnet.

Variabelnamn	Position	Värde
värde1	0	10
värde2	1	11
värde3	2	15

För att komma åt siffrorna skriver man exempelvis värde[0], värde[1], värde[2] osv. Ska man exempelvis räkna ut summan av dessa värden kan man köra en loop som läser av och summerar dem.

```
int sum = 0;
for (int add = 0; add < värde.Length; add++) { sum += värde[add]; }
```

Detta är effektivt om man har ett hundratal värden att summera, istället för att skriva
sum=värde1+värde2+...+värde99+värde100;

Arrays är statiska i sin storlek, om du definierar att en array ska innehålla 10 värden, så kan du inte förstora upp den till att innehålla 12 värden. Du får skapa en ny array helt enkelt.

```
int[] värde = new int[10]; // Kan enbart ta emot 10 värden
```

Man kan göra arrays av alla typer av data som finns i C#, även av dina egna klasser.

```
FilmLista[] Filmer=new FilmLista[5];  
Kom baras ihåg att varje rad i din array måste instansieras.  
Filmer[0]= new FilmLista();  
Därefter kan du nå egenskaperna i din class enligt  
Filmer[0].Namn="Indiana Jones 4";  
Filmer[0].Betyg=2;
```

Se exempel [Arrayer](#) och [Handellista](#)

POSITION VS ANTAL

En lurig sak med arrayer är att innehållet placeras i positioner numrerade från 0 och uppåt. Vilket betyder att om man lagrat ett värde i position 0, så kommer antalet värden lagrade att vara 1. Vilket är logiskt.

Man kan belysa det såhär

Position	Total Antal poster	Värde
0	1	"Hej"
1	2	"Hallå"
2	3	"Tjabba"

Antalet poster i arrayen är 3, men den sista posten finns i position 2 och den första finns i position 0.

Om man vill ha sista värdet i arrayen så får man alltså läsa av **[array.length-1]**.

Se kodexempel [Arraypositioner](#)

TVÅDIMENSIONELLA ARRAYER

Har du använt Excel så vet du hur det fungerar att arbeta i ett rutnät. Ibland vill man lagra information i flera rader och kolumner. Om man arbetar med kartor eller koordinater.

```
int[,] värde = new int[5,5]; // tar emot 5 rader, 5 kolumner
```

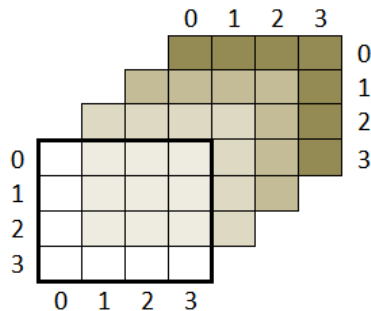
	0	1	2	3	4
0					
1					
2					
3					
4					

I koordinatsystem talar man om X och Y position, detta tänkande fungerar på tvådimensionella arrays, där man lagrar värdena i rader och kolumner. Ys värde motsvarar en rad, Xs värde motsvarar en kolumn. Man kan använda detta när man håller reda på exempelvis kartpositioner. Ett annat användningsområde är exempelvis att varje rad motsvarar en artist och kolumnen motsvarar poängen de fått av olika domare.

FLERDIMENSIONELLA ARRAYER

Man kan skapa 3dimensionella arrayer:

```
int[, ,] treDimensionell = new int[4, 4, 4];
```



Ett sådant upplägg av en tabell är kubformad, man håller alltså reda på höjd, bredd och djup (X,Y och Z). Självklart kan man lägga in fler dimensioner och högre antal rader och kolumner.

ABSTRAKTION

Abstraktion är att göra saker till en enklare form. Exempelvis när vi tittar på motorvägen lägger vi märke till att det är en massa bilar som kör omkring på den. Vi ser inte alla skruvar, muttrar, motordelar, slangar, plåt och annat som bilarna är gjorda av. Vi ser en enklare form, det är så mycket information som vi behöver. Skulle vi behöva mer information så kanske vi ser bilen enligt dess märke och tillverkningsår. Då får man reda på betydligt mer information än om man bara säger "en bil".

Man gör på samma sätt när man programmerar, antingen bygger man upp abstrakta classer som talar om de vanligaste egenskaperna för en klass, eller så gör man en interface med specifika egenskaper.

Abstraktionen används just för att kunna gruppera klasser och typer i en specifik behållare. På så sätt kan man handskas med de gemensamma egenskaperna i en och samma metod.

LIST<T>

En List<> objekt förser oss med ett enklare sätt att handskas med listor och datamängder. List classen är **generisk** (det ser man av <T> i beskrivningen)

Listans storlek är dynamisk till skillnad från Arrayen som har fast storlek. Listan kan innehålla precis vad som helst, vad den ska innehålla anger man vid instansieringen av listan. List classen har ett antal standard metoder som vi har tillgång till.

Vissa av metoderna är markerade som krångliga, för att de kräver Lambda kodning och det är inte förklarat i häftet än. Det kommer senare 😊

Metod	Förklaring
Add	Lägger till en post i listan
AddRange	Importerar en Array till listan
BinarySearch	Söker igenom listan och returnerar en int som index till värdet man söker
Capacity	Sätter eller returnerar mängden data den kan innehålla innan listans array måste skapas om
Clear	Raderar allt i listan
Contains	Söker efter viss data och returnerar en bool, true om det finns och false om det inte finns
ConvertAll	Castar om alla objekt i listan och returnerar en ny lista med de nya värdena
CopyTo	Kopierar hela listan till en array
Count	Returnerar antal objekt i listan
Exists	Söker igenom listan för att se om ett visst objekt finns (lite krångligare än Contains)
Find	Söker igenom listan efter en specifik post (krånglig)
FindAll	Söker igenom listan och returnerar en lista med alla objekten som matchar (krånglig)
FindIndex	Söker igenom listan och returnerar en int med index till den matchande posten (krånglig)
FindLast	Söker igenom listan och returnerar sista posten som matchar sökningen (krånglig)
FindLastIndex	Söker igenom listan och returnerar en int med det sista matchande index.
ForEach	Loopar igenom listan och anropar en metod som får ta hand om värdet som hittas (se kodexempel)
GetEnumerator	Om man har en class med lista, som implementerar IEnumerable så kan man returnera Listans inbyggda enumeratorn och därmed slippa skapa en extra class för att implementera IEnumerator .
GetRange	Returnerar en lista med posterna som finns i det antal och från det startvärde man angett.
IndexOf	Returnerar index positionen för det värde man söker
Insert	Lagrar ett värde, vid en specifik position i listan.
InsertRange	Lagrar en serie värden, från en specifik position och framåt i listan. Man kan exempelvis skicka in en array
lastIndexOf	Sista positionen för sökt värde
Remove	Tar bort en post från listan
RemoveAll	Tar bort alla matchande poster i listan (krånglig)
RemoveAt	Tar bort post vid en specifik position i listan
RemoveRange	Tar bort antal poster från en viss position i listan och framåt
Reverse	Ändrar posternas ordning så att första posten kommer sist och sista posten kommer först, från den angivna positionen och fram.
Sort	Sorterar listan
ToArray	Returnerar en array med hela listan
TrimExcess	Packar ihop listan för att spara minne

Som du ser så har man lite mer verktyg att leka med i jämförelse med Arrays, dock ska man komma ihåg att det är en class som använder arrays, prestandaskillnaden är minimal, men den finns. Är man tvungen att göra tidsprecisa metoder så är det kanske bättre att använda Arrays. I annat fall är List<> ett utmärkt verktyg för datalagring.

ABSTRAKTA CLASSER

En abstract class är en class som egentligen inte innehåller kod, den innehåller metodhuvuden för att beskriva publika standardmetoder. Dessa standardmetoder måste implementeras i klassen som implementerar den abstrakta klassen. Den abstrakta klassen kan inte instansieras utan måste implementeras av en annan class först.

De metoder som deklarerats som abstrakta får inte vara privata, de får däremot vara publika, protected eller internal.



Man kan likna det med att en abstrakt class är en form för en pepparkaksgubbe, den visar hur gubben ska se ut, men den är inte ätbar. Själva pepparkaksgubben (som implementerar formens egenskaper) är ätbar däremot. Formen talar inte heller om vilka ingredienser (privata metoder) din pepparkaksdeg ska innehålla.

En abstract class kan ärva från en vanlig class, men den kan ändå inte instansieras. De metoder som deklarerades i superclassen förblir deklarerade även i den abstrakta classen. När den abstrakta classen subclassas kommer metoderna att förbli oförändrade såvida inte någon gör en **override** på dem. Dock måste alla abstrakta metodhuvudena som angavs i den abstracta classen implementeras innan programmet blir körbart.

Man implementerar classen genom att "ärva" den och sedan skapa alla metoder som den abstracta classen vill ha.

Observera att classen definierar en publik bild av hur den ska se ut, den talar inte om vilka publika variabler eller metoder som ska finnas i din class.

Se kodexempel **Abstrakt class**

INTERFACE

Ett interface liknar på många sätt en abstrakt class, men den har vissa skillnader.

Om en abstrakt class är en pepparkaksform, så är en interface en egenskap som ögon eller färg. De classer som instansierar den abstrakta classen tillhör den specifika sorten, men flera olika sort kan implementera en interface.



Alltså kan pepparkaksgubbar ha ögon, och pepparkaksgrisar (som har en annan form) också ha samma sorts ögon. Även om de inte har samma ursprungsform.

Interface kan ärva från andra interface, men inte från classer.

Klassen som implementerar interfacet kan implementera flera interfacen, och ändå ärva från en annan class. Dock kan man bara ärva från bara en class.

En viktig språkregel (enligt Microsoft Standard) är att namnet på ett interface alltid börjar med **I** och slutar med **able**, exempelvis **IComparable**, **IEnumerable**, **IDisposable** mm. Namnet ska alltså antyda att classen kan göra en viss sak. När man läser namnet **IComparable** ska man alltså få intrycket av att det står "class is able to compare".

Se kodexempel **Interface**

ICOMPARABLE

IComparable används för att jämföra storlek mellan classer. Den returnerar alltid en int enligt följande mall.

-1 = Det andra värdet som jämförs är större

0 = Båda värden är lika

1 = Det andra värdet som jämförs är mindre

I sin enklaste form ser implementeringen av **IComparable** ut såhär.

```
public int CompareTo(object obj)
```

```
{
    Return return this.myValue.CompareTo((MyClass) obj).myValue)
}
```

Det fungerar men risken finns att hela programmet krashar. Det finns vissa grundregler man ska tänka på när man implementerar IComparable.

- 1) Kontrollera alltid om objektet som skickas in är null, returnera felmeddelande eller valfri värde (-1 om du vill att null ska räknas som större, eller 1 om det ska räknas som mindre)
- 2) Kontrollera att objektet som skickades in är av samma sort som klassen vi arbetar med. (Se **Is**)
- 3) Om det är ett objekt du får in, **typecasta** den innan jämförelsen kan göras
- 4) Om typen du vill jämföra med redan implementerar IComparable, använd den färdiga metoden istället för att göra om hela arbetet.

Se kodexempel **Icomparable**

ICOMPARABLE<T>

Detta fungerar på samma sätt som IComparable men här får man en instans av den valda klassen istället för ett objekt. Därmed slipper du kontrollera om den är av rätt sort, och du slipper casta den till rätt sort av class.

Dock måste man fortfarande kontrollera om den är null, och sedan jämföra de värden man vill jämföra.

Se kodexempel **Icomparable<T>**

IEnumerable

IEnumerator gör att klassen blir kompatibel med foreach. Det enda som Interfacet kräver är att man ska returnera en instans av en **class som implementerar IEnumerator** och att den har tillgång till de värden som den aktuella klassen (som ska köras igenom foreach) har.

```
public IEnumerator GetEnumerator()
{
    return new Iterator(this);
}
```

IEnumerator

IEnumerator används i en class som har tillgång till en datasamling, exempelvis en array, denna har tre standardmetoder. Man har ett indexvärde (ex `int index = 0;`) för att hålla reda på den aktuella positionen i arrayen där man befinner sig.

Se exempel **Ienumerator**

IEnumerator<T>

När man deklarerar Ienumeratorn kan man ange vilken sorts data den ska returnera. Då slipper man typecasta när man tar emot informationen (foreach gör det automatiskt förvisso).

GENERISKA KLASSER

Generiska klasser är classer som inte är **typbestämda**. Man ger den en typ när man instansierar den. Man känner igen en generisk class på grund av symbolerna <> framför.

Man kan skapa egna generiska klasser genom att lägga till <T> efter classnamnet.

```
class MinGeneriskaClass<T>
```

Sedan använder man T som en vanlig typ, som bestäms av den som instansierar classen.

Interfaces kan också vara generiska, det ser man bland annat på interfacen **IComparable** och dess generiska motsvarighet **IComparable<T>**.

Se exempel **Generisk class**

ALGORITHM

En algoritm är en serie instruktioner som beskriver hur en specifik beräkning eller databehandling ska gå till. Inom datavärlden kallas det en funktion eller en **metod**.

Se exempel **Algoritm**

EXCEPTIONS

Exceptions är de undantagsfall som kan ske när programmet körs. Om man inte använder sig av exceptions får man ta till if-satser och andra kontroller för att se till att programmet inte kraschar. Exceptions i sig är inte lösningen på problemet med krascherna, de är objekt med informationen man behöver för att kunna stoppa programmet från att krascha.

TRY CATCH

När man vill kontrollera om ett fel kan ske, skapar man ett kodblock av kommandot try. Nu ska man inte tro att det löser alla problem. Att ha en try{} block som täcker en hel metod är ganska meningslöst, för man lär inte få reda på var i metoden det fick fel. Try använder man alltså på just de delarna av koden där det kan smälla. Exempelvis vid beräkningar.

Man kan ha flera catch block efter varandra, man tar alltid den minst specifika sist. Uppbyggnaden är på sätt och vis som en **Switch case** sats.

Man kan ha try catch block inom andra try catch block, det fungerar bra.

```
try
{
    //Utför beräkning
}
catch (ArgumentNullException)
{
    Console.WriteLine("Du skickade in ett nullvärde!");
}
catch (ArgumentOutOfRangeException)
{
    Console.WriteLine("Du skickade in ett för stort värde");
}
catch (ArgumentException)
```

```

{
    Console.WriteLine("Du skickade in ett felaktigt värde");
}
catch (Exception ex)
{
    Console.WriteLine("Något gick fel:{0}", ex.Message);
}
finally
{
    //Städa upp i minnet, detta körs oavsett om det blivit fel eller inte
}

```

FINALLY

Finally blocket körs oavsett om det blivit fel eller inte.

LITE ATT TÄNKA PÅ

Det finns massor med färdiga exceptions man kan använda, finns inte det exception man vill ha så kan man skapa egna classer för det (men det gör vi en annan gång). Att tala om för användaren att något fick fel är kanske inte alltid den bästa lösningen. Kan man fixa till felet så gör man det i antingen catch eller finally.

Man kan även fånga in exceptiondatan i en variabel `catch (Exception ex)` "och där få lite mer info om vad som gått snett.

Om din metod inte har en try-catch kontroll, men metoden som anropat den har en try-catch, så kommer den att fånga upp felet. Det är bra förvisso att felet fångas, men det kan leda till att du får helt felaktig felmeddelande i din kod – på grund av att felmeddelandet orsakades av en annan metod.

Om ingen metod har en try-catch block så kommer felet att fångas upp av kompilatorn som kommer i sin tur att meddela att det blivit fel, och avsluta ditt program.

....MEN

Även om en try-catch kodblock kan få en att känna sig tryggare så ska man tänka på att det är tidskrävande för programmet. Man märker inte det om man kör vanliga program, men när man programmerar i miljöer som kräver tidsprecision (exempelvis spel) så ska man nog ta det lugnt med try-catch.

If satser fungerar snabbare och har man tänkt igenom koden bra så klarar man sig med få try-catch.

LOOP

En **loop** är en rutin som utförs upprepade gånger enligt ett mönster man gett programmet. Exempelvis att den ska utföra samma sak 10 gånger, eller tills ett visst villkor uppfylls. Allt som finns inom den angivna loopen kommer att upprepas. Det är som att säga till datorn (se **pseudokod**):

- Addera summan av angivna tal tio gånger (**for loop**)
- Minska ett värde tills den är mindre än 7.5 (**while loop**)
- Fråga om ålder tills du hittar en person över 18 år (**do loop**)

OÄNDLIG LOOP

En oändlig loop är något man vanligen undviker. Om loopen inte har en kontroll för att avbryta vid nödfall kan programmet låsa sig. Det är inte trevligt! Tänk igenom dina villkor innan du kör igång loopen så att du inte riskerar att hamna där.

Det kan ske om du till exempel ska räkna ner i en loop, men skriver ++ i din forsats istället, eller som du har en while sats som inte ändrar sitt kontrollvärde.

Om du inte har full koll på dina variabler, så är det bättre att kontrollera om ett värde är större eller mindre än kontrollvärdet, än att kontrollera om den har samma värde som kon kontrollen.

Se exempel [Oändliga loopar](#)

FOR LOOP

En for loop är en loop där man anger hur många gånger en rutin ska upprepas. Dess struktur är enkel och den är bra att använda när man vet exakt hur många gånger upprepningen ska ske.

I pseudokod skulle man kunna uttrycka det som

UPPREPA 10 gånger

Beräkna en massa siffror

Först skriver man nyckelordet

Loopens egenskaper anges inom parentes

Sedan anger man variabeln den ska använda som räknare, i vanliga fall deklarerar man den här och tilldelar den startvärde.

Varje satsdel separeras med ett semikolon

Sedan skriver man villkor för att upprepa loopen

Sedan anger man hur den ska förändra räknaren

Slutligen stänger man parenteser

Sedan lägger man till koden som ska upprepas inom måsvingar

```
For
(
  int räknare = 0
;
  räknare < 10;
  räknare++
)
{ }
```

Se kodexempel [For-sats](#)

FOREACH

En `foreach` loop är lite annorlunda än en for loop. Här anger man bara vilken typ man ska arbeta med, var den ska lagras temporärt och från vilken samling av data man ska utgå ifrån.

Exempelvis om man har en `array` av strängar och vill loop igenom allihopa, så skapar man en foreach loop som går igenom alla strängarna.

```
foreach (string str in stringCollection)
{
    //Lek med strängarna
}
```

```
}
```

Man bygger upp satsen enligt detta mönster

```
foreach  
String
```

```
str
```

```
in
```

```
stringCollection
```

Kommandot **foreach**

Typen man ska arbeta med, exempelvis string, int eller vad nu det är för typ din array innehåller

Namnet på din temporära variabel. Den kommer att skrivas över varje iteration (alltså varje gång som loopen kör en runda)

Kommando för att skilja den temporära variabeln och arrayen eller datasamlingen

En array, list eller anna datasamlingsobjekt som implementerar **IEnumerable**.

Man kan göra likadant med olika typer och olika sorts datasamling. Det enda kravet är att samlingen av typer ska implementera interfacet **IEnumerable**. Med andra ord vill du att din klass (som innehåller en samling data) ska fungera med foreach så får du helt enkelt implementera **IEnumerable** interfacet i din class. Men mer om det sen. Huvudsaken är att foreach fungerar med bland annat arrays och List objekt, och med det kan man komma långt.

Ovanstående foreach sats skulle kunna skrivas om till en vanlig for-sats

```
for (int tmpInt = 0; tmpInt < stringCollection.Length; tmpInt++)  
{  
    string readonly str = stringCollection[tmpInt];  
    //Lek med strängarna  
}
```

När man ser det så, får man medge att foreach har sin charm... eller hur?

Det finns en nackdel med foreach dock, objekten man arbetar med, alltså den temporära variabeln, blir skrivskyddad. Detta på grund av att variabeln är egentligen en polymorfad **IEnumerable** och dess egentligen värden är dolda för foreach-satsen.

Man löser detta problem på två sätt. Antingen att man skapar en ny lista, där man matar in alla värden man får fram i foreach satsen (plus de modifikationer man vill göra) och sedan ersätter den ursprungliga listan, eller så ändrar man inte i objekten och använder foreach satsen enbart för att göra utskrifter eller anropa metoder inom den classen man foreachar.

WHILE LOOP

En while loop är en loop som upprepas så länge ett villkor uppfylls. Exempelvis i pseudokod skulle man kunna säga:

SÅ LÄNGE *det finns myggor*
Döda myggorna

Tänk på att **variabeln** som ska användas i jämförelsen för while-satsen måste deklareras innan while-satsen. Glöm inte heller att påverka din variabel så att du inte får en **oändlig loop**!

Först skriver man nyckelordet	<code>While</code>
Loopens egenskaper anges inom parentes	<code>(</code>
Sedan skriver man villkor för att upprepa loopen	<code>räknare < 10</code>
Slutligen stänger man parentesen	<code>)</code>
Slutligen lägger man till koden som ska upprepas inom måsvingar	<code>{ }</code>

Se kodexempel [While-sats](#)

DO LOOP

En do loop är i mångt och mycket som en while loop, som upprepas så länge ett villkor uppfylls. Exempelvis i pseudokod skulle man kunna säga:

UPPREPA

Döda myggorna
OM *det finns myggor kvar*

Tänk på att **variabeln** som ska användas i jämförelsen för while-satsen i slutet måste deklareras innan do-satsen. Glöm inte heller att påverka din variabel så att du inte får en **oändlig loop**!

Först skriver man nyckelordet	<code>do</code>
Sen lägger man till koden som ska upprepas inom måsvingar	<code>{ }</code>
Därefter skriver man nyckelordet	<code>While</code>
Loopens egenskaper anges inom parentes	<code>(</code>
Sedan skriver man villkor för att upprepa loopen	<code>räknare < 10</code>
Slutligen stänger man parentesen	<code>)</code>
Och avslutar med semikolon	<code>;</code>

Se kodexempel [Do-sats](#)

NÄSTLADE LOOPAR

Nästlade loopar är helt enkelt loopar som ligger i varandra. Om man kanske vill skriva ut värden från en **tvådimensionell array** så måste man gå igenom en rad i taget, och i varje rad gå igenom en kolumn i taget.

Då kan man lösa det med en loop i en annan loop. Den första loopen går igenom varje rad, och när den valt en rad så kommer nästa loop att gå igenom alla kolumner i den. Ett annat sätt att tänka sig en nästlad loop är att titta på en analog klocka.

Klockans timvisare rör sig ett steg (eller en rad om man tänker enligt förra exemplet), för var steg som den rör sig kommer minutvisaren att gå femtionio steg (eller femtionio kolumner om man tänker enligt exemplet ovan). Därefter flyttar sig timvisaren igen och minutvisaren börjar på noll.

Timvisaren representerar den yttre loopen i exemplet, och minutvisaren representerar den inre loopen.

Se exempel Nästlade loopar.

SUBROUTIN

En **metod** i ett program, även kallad procedur, funktion eller underprogram. Det är alltså en kodsekvens i programmet som är avskilt från grundkoden men som används via anrop.

Se kodexempel **Metodanrop**

OOP – OBJEKTORIENTERAD PROGRAMMERING

Objektorienterad programmering är ett sätt att programmera där man använder sig av olika objekt som kommunicerar och påverkar varandra. Man kapslar in programdelar (**Class**) för att de ska fungera som om de vore oberoende av grundprogrammet, och man förser dem med **egenskaper**. Man skapar (**instansierar**) **objekt** av dessa classer och använder sig av dessa för att få programmet att likna det verkliga livet. Exempelvis kan man ha en bil-class och förse den med fyra instanser av hjul-class. På så sätt kan var och en av hjulen behandlas som separata objekt, även om de är en del av bilen.

ARV

Arv är ett sätt att återanvända classer och förse dem med nya egenskaper och metoder som skiljer den från originaluppsättningen. Istället för att återuppfinna hjulet låter man classen kopiera egenskaperna från basclassen. Exempelvis om man skapar en hund-class. Den är en standard hund-class med alla standaregenskaper en hund ska ha. När man sedan vill skapa en Chihuahua och en pitbull får man förse dessa med speciella egenskaper som bas-hunden kanske inte har. Alltså skapar man en Chihuahua class och en Pitbull class, båda ärver från hundclassen. På så sätt slipper man skapa om hela hunden för var ras man gör. Både pitbullen och Chihuahuan kommer att ha alla egenskaper en bas-hund har, plus sina speciella egenskaper.

I kodexemplet gör vi att Pitbullen ska vara gosig och Chihuahuan ska vara bjäbbig, och då pudeln ärver från Chihuahuan så är den också bjäbbig. Pitbullen kommer dock aldrig att ha egenskapen bjäbbig då den ärver från Bas-hund och där finns inte Bjäbbig egenskapen. Det innebär också att pudeln och chihuahuan inte kommer att vara gosiga heller, då den egenskapen finns bara hos pitbullen.

En class kan ärva från en annan class, men bara från **en** class.

Se kodexempel [Hund-class](#)

BASE

Base är ett objekt som pekar på superclassen. Den finns endast tillgänglig i den egna classen och kan inte skickas som parameter, varken in eller ut. Base används för att anropa metoder och variabler i superclassen.

Exempelvis

```
Public override void SkrivUtInfo()  
{ base.SkrivUtInfo() }
```

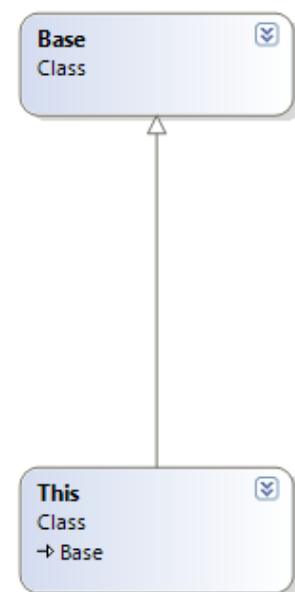
Se kodexempel [Base och this](#)

THIS

När man ska kommunicera med variabler och metoder inom den egna classen, men vill ändå känna sig säker på att det sker inom den egna classen använder man objektet this. Detta objekt är den instansieringen av classen som anropats för tillfället.

Man kan även ange this som in eller return parameter.

Se kodexempel [Base och this](#)



IS

Is är ett sätt att ta reda på om ett objekt eller en instansierad class tillhör en viss typ.

```
if (frukter[0] is Apple) {...}
```

det är egentligen samma sak som att skriva

```
if (frukter[0].GetType() == typeof(Apple)) {...}
```

men att använda **is** är enklare och ger samma resultat. Is är inte känslig för nullobjekt heller, if satsen med GetType() orsakar däremot ett fel om objektet man vill jämföra med är null. Så Is är alltså säkrare att använda.

VIRTUAL

En virtuell metod är en metod som kan återskapas vid arv. Med andra ord, om superclassen har en virtuell metod så kan subclassen välja att skapa en ny version av metoden. Detta görs med modifieraren **Override**. Classen som overridear metoden måste skapa en metod vars metodhuvud är exakt likadan som den virtuella.

Man deklarerar det helt enkelt såhär

```
public virtual int vikt() {...}
```

Se kodexempel **Sealed**

OVERRIDE

När en subclass ska ersätta en metod från superklassen anger man modifieraren override, detta innebär att när man anropar metoden så kommer den nya deklareringen av metoden att anropas, istället för superklassens originalversion. Vill man tvunget anropa superklassens version så kan man lägga till **base.metod()** och då anropas superklassens metod. Detta kan man bara göra ifrån subclassen.

En virtuell metod som deklarerats om med override kommer att fortsätta att vara virtuell, så virtual modifieraren behöver inte anges igen.

```
public override int vikt() {...}
```

NEW

Om man vill frånga från arvskopplingen så kan man ange modifieraren new. Därmed bryts kopplingen till superclassen. Dock kan base.metod() fortfarande anropas. Ibland vill man göra detta för att bryta kontakten med variabler som lagras i superclassen, och ersätta dem med egna värden som inte påverkar superclassen i sig. Ett annat tillfälle det kan vara bra att ha är när man har statiska värden eller metoder lagrade i superclassen.

Förutom när man har med statiska värden att göra, så märker man inte av skillnaden mellan new och override förrän man ger sig på polymorfism. Metoden markerad med new kommer inte att hittas av polyformismen, istället kommer metoden från superclassen att anropas!

```
public new int vikt() {...}
```

SEALED

När man vill sätta stopp för arvet kan man försegla ärvandet (seal) och därmed hindra att subclasser overridear metoden. Subclassen kan "overridea" metoden med new eller genom att inte skriva override alls, det kommer att fungera men polymorfismen sätts ur spel. Med andra ord kommer polymorfismen att **inte** hitta metoden och istället anropa superclassens metod, i detta fall metoden som blev markerad med sealed.

Se kodexempel [Sealed](#)

GET

Get-metod är en metod som returnerar ett värde från en **class**, om classen har just det **fältet** som **privat**. En get metod tar inte emot parametrar, den returnerar bara.

Se exempelkod [Get/Set metoder](#)

SET

Set är motsatsen till get, den tar emot en parameter som innehåller det värdet som ska stoppas in i den privata variabeln i classen. Denna metod returnerar oftast ingenting. I denna metod har man ett perfekt tillfälle att lägga in kontroller för vad som skickas in, om man exempelvis inte vill att värdet som skickas in ska vara större eller mindre än ett specifikt värde.

Se exempelkod [Get/Set metoder](#)

EGENSKAPER (GET/SET)

Man kan ge egenskaper till en class, då använder man sig av get/set metoden, fast i en lite annorlunda form än de förra exemplen. I de förra exemplen användes metoder för att lösa get/set problematiken, man kan dock göra det med en speciell struktur för detta. Oftast namnger man dessa med samma namn som variabeln, fast med versal i första bokstaven.

Först väljer man åtkomstgrad	<code>[private public osv...]</code>
Sedan anger man om den är statisk	<code>[static]</code>
Därefter anger man variabeltyp den ska returnera	<code>[void int string osv...]</code>
Sedan ger man namn på egenskapen (inga parenteser)	<code>NamnPåEgenskapen</code>
Sedan skriver man in strukturen	<code>get{return mittVärde};</code> <code>Set{mittVärde = value};</code>

I get delen av strukturen skriver man in vad som ska hända när användaren av classen vill se vad som finns lagrad i det privata fältet som egenskapen handlar om.

Man tilldelar värden till egenskapen på samma sätt som man tilldelar värden till en variabel, och man läser av värdet på egenskapen på det sättet också.

I set delen av strukturen skriver man in vad som ska hända när användaren av classen vill skriva in ett nytt värde, är det värdet godkänt? Passar den in i din class och hur du vill att den ska bete sig? Om allt är OK då sparar du `value` variabeln i din variabel.

Se exempel [Egenskaper](#)

SKRIVSKYDDADE EGENSKAPER

Man kan skrivskydda en egenskap om man bara vill att användaren ska få se värdet på fältet, men inte kunna ändra i det. Man gör så enkelt som att man inte skapar något `set` metod.

Se exempel [Annorlunda Egenskap](#) (som använder en skrivskyddad egenskap)

"FEJKADE" EGENSKAPER

Man kan skapa egenskaper som inte returnerar värdet från ett fält, utan snarare från en beräkning. Exempelvis om man vill beräkna värdet på glassarna i classen `Glasskiosk`. Då kan man skapa en egenskap som helt enkelt returnerar antal Pucko multiplicerat med priset, plus antal X15 multiplicerat med priset.

I kodexemplet använder man fältet med födelsedata för att räkna ut åldern på personen.

Se exempel [Annorlunda Egenskap](#)

VALUE

`value` variabeln är en variabel som skapas automatiskt när man använder egenskaper.

POLYMORFISM

Polymorfism är en sak som kan ta flera olika former. Inom programmering betyder det att en samling subclasser kan lagras i en variabel deklarerad att innehålla instanser av superclassen.

```
Fordon[] minaFordon = new Fordon[3];
minaFordon[0] = new Cykel();
minaFordon[1] = new Bil();
```

Alla de gemensamma metoderna i superclassen `Fordon`, som antingen ärvt eller **Overridats** av subclasserna, kan man använda i ett gemensamt gränssnitt för att bearbeta de olika sorters subclasser som skapats. På så sätt slipper man göra en speciell metod för var typ.

Man kan använda samma teknik när man implementerat **Interfaces** eller **Abstrakta classer**.

Polymorfism är alltså tekniken att använda flera olika sorts classer, med gemensamma drag, i ett och samma gränssnitt.

De metoder och egenskaper som inte ärvs utan lagts till subclasserna, kommer inte att finnas tillgängliga när man arbetar med polymorfism. Vill man nå dessa specifika typer måste man **casta** objekter först.

Se exempel [Polymorfism](#)

NAMNGIVNINGSKONVENTIONER

UNGERSK NOTATION

Ungersk notation är ett sätt att namnge variabler för att ge en antydning om vad de är för typ. Detta har använts sedan 80-talet, då Charles Simonyi uppfann det. Det går helt enkelt ut på att man anger en eller tre bokstäver av typens namn framför variabelns namn.

Man skriver typen med gemener och variabelnamnet med versal i första bokstaven av ordet (då man inte kan använda mellanslag).

Ungersk notation 1 tecken	Ungersk notation 3 tecken	Förklaring
sLoginNamn	strLoginNamn	En sträng som ska innehålla ett namn
iHögstaHastighet	intHögstaHastighet	En int som ska innehålla värdet för högsta hastigheten
dPI	dblPI	En double som innehåller PI

Det används även i metodnamn ibland, exempelvis `btnClick()` som skulle kunna vara en metod som anropas när en knapp trycks ner.

Man använder ungersk notation oftast i språk som ASP classic, javascript, vbscript, PHP och annat skoj. Dessa språk kräver inte typdefinieringar explicit som C# gör.

CAMEL NOTATION

I Visual Studio får man väldigt mycket hjälp att hålla reda på vilka variabler man använder och vad de är för typer. Man kan se det genom att peka på dem, eller genom att välja dem i en class exempelvis. Då får man reda på namnet och typen. Därför har man gått över till Camel notation.

Ungersk notation 1 tecken	Camel notation
sLoginNamn	loginNamn
iHastighet	högstaHastighet
dPI	Pi

Om det är en lokal variabel så börjar man med gemener, exempelvis `loginPassword`.

Camelnotation använder man mest till **variabelnamn**.

PASCAL NOTATION

Det är mer eller mindre samma sak som Camel Notation, men man börjar alla ord med versal. Pascal notation används även inom Kemin, tänk bara på vanlig koksalt, Natriumklorid, dess kemiska formel är NaCl (Natrium Chloride, även kallad Sodium Chloride). Om man tänker på det så, så känns hela systemet ganska naturligt.

Ungersk notation 1 tecken	Camel notation	Pascal notation
sLoginNamn	loginNamn	LoginNamn
iHögstaHastighet	högstaHastighet	HögstaHastighet
dPi	Pi	Pi

Pascalnotation används mest till **Metodnamn**.

ÖVRIG INFORMATION

Här finns extra information som är bra att känna till

KODLÄGEN

Det finns två lägen i programmeringen. Runtime och Designtime. Dessa skiljer sig på många sätt.

En viktig regel att komma ihåg är:

DET ÄR ALLTID BÄTTRE ATT FÅ KOMPILATORFEL I DESIGNTIME ÄN ATT FÅ RUNTIMEFEL!

Kompilatorfel och designtime-fel kan man alltid fixa. Runtimefel visar sig kanske inte förrän användaren gör ett fel, och då har man problem.

Lärdomen är därmed.

- 1) Skriver du fel kod och kompilatorn säger till dig, var glad att den sa till och att din framtida användare slipper bli ledsen på dig!
- 2) Skriv kod med bra felhantering, lämna inte koden åt slumpen!

DESIGNTIME

Designtime är när man programmerar utan att köra programmet.

Designtime är när man programmerar i vanliga fall. Man kan då ändra i koden hur man vill, ändra utseendet på formulärer, döpa om variabler och classer. Man kan praktiskt taget göra vad som helst. Visual Studios editor förser en med en massa specialverktyg för att kunna ändra och editera.

RUNTIME

Runtime är när man kör programmet och debuggar.

Här har man vissa hinder (i asp.net kan man bara ändra html koden). Man kan inte ändra lika mycket, variabelnamn och classer är lite känsliga att ändra i. Drastiska förändringar i koden är inte tillåtna. Däremot kan man se hur koden arbetar i realtid och man kan följa variablernas värden.

TYPER I MINNET

När vi skickar över variabler till eller från en metod så sker det på olika sätt beroende på vad det är för parameter vi skickar. Detta är viktigt att känna till, för att det påverkar hur informationen bearbetas och vad som behöver returneras.

VALUE TYPE (BYVAL)

När en variabel är value type betyder det att när man skickar iväg den som parameter, så skickas en kopia av den till metoden.

Om vi har en metod som tar emot en int

```
void BeräknaVärden(int y) { /* gör beräkningar */ }
```

och anropar den med

```
int x=5;  
BeräknaVärden(x);
```

så skickas en kopia av värdet 5 till metoden. Det värdet den får in i variabeln y kan bearbetas och förändras hur man vill, det påverkar inte variabeln x i huvudmetoden på något sätt. Vill man komma förbi denna begränsning kan man göra x till en **ref** variabel.

Alla inbyggda typer som int, double, long, decimal, string, enums och **structs** är By value variabler. Deras värden kopieras över till metoden som tar emot eller från metoden som returnerar. Är det stora mängder som ska skickas över (exempelvis en jättestor sträng) så kan det slöa ner programmet.

Se exempel **ByVal och Byref**

REFERENCE TYPE (BYREF)

Raka motsatsen till By value, istället för värdet på variabeln så skickas en minnesreferens. Detta innebär att alla förändringar som görs på den skickade variabeln kommer att påverka originalvariabeln som användes vid anropet. Detta kan få förödande effekter om man inte är medveten om vad som händer bakom koden.

Alla objekt, instansieringar av classer och egna typer är "by reference". Tänk på att arrays är också by reference, du kan alltså ha en array med int (som är byval i sin grundform) och skicka till en metod, som ändrar värdet på dina int i arrayen (arrays är byref). När du kommer tillbaka till metoden därifrån anropet skedde så kan din Array ha förändrats!

Se exempel **ByVal och Byref**

TYPSÄKRING

Att "typa en class" eller ett interface är att låsa fast en class till en viss typ. Detta kan vara exempelvis om man har en objekt array. I den kan man egentligen stoppa in vilket objekt som helst.

```
object[] objektLista=new object[3]();
objektLista[0]="Hello";
objektLista[1]=5;
objektLista[2]=new Random();
```

Ibland kan det vara bra att ha det, eller om man gör en array av en superclass och stoppar in olika sorts subclasser i den.

Man kan göra det, men det är kluddigt, speciellt när man vill veta vad som finns var i arrayen. Alltså låser man hellre arrayen till att innehålla en viss sorts typ, och därmed gör man den typsäker (man typar den).

```
//Typad array till int
int[] minaTal=new int[10];

//Typad array till sträng
string[] minaSträngar=new string[10];
```

Detsamma gäller när man implementerar vissa **Interfaces** som **Comparable**, om man inte typar den kommer implementeringen att se ut såhär.

```
public int CompareTo(object obj)
{
}
```

Då kommer jämförelsen att ske mot alla tänkbara objekt kompilatorn kan tänkas vilja jämföra din class mot. Vill du vara säker på att objektet som jämförs är av samma sort som din class måste du antingen typa den till den classen du vill jämföra mot, eller så får du jämföra med **GetType** eller **is** metoden.

Om du däremot typsäkrar interfacet till "MinClass", **Comparable<MinClass>**, så kommer jämförelserna att ske enbart med "MinClass".

```
public int CompareTo(MinClass other)
{
}
```

Detta är mer lätthanterligt.

Ha det som regel, typsäkra allt du kan typsäkra. Då slipper du problem med **casting** och **is** kontroller.

CASTING

Casting är när man byter typ på ett objekt eller typ. Om man har exempelvis en int som man vill göra om till en long kan man skriva.

```
int tal=5;
long storTal=tal; //automatisk casting
```

Här kan kompilatorn se att en int ska bli en long, då en int aldrig kan bli större eller mindre än en long så kommer kompilatorn att omvandla det rakt över. Om vi nu ska gå tillbaka från long till int så vet inte kompilatorn längre om talet kan få plats i en int. Vi kan forcera kompilatorn genom att forcera fram en casting. Det gör vi genom att skriva typen inom parentes.

```
tal=(int) storTal;
```

Man kan göra på samma sätt när man arbetar med arv och polymorfism. Om vi har en superclass kallad Fukt och låter klassen Äpple och Päron ära ifrån frukt så kan vi lagra dem i en Fukt instans (se polymorfism).

```
Fukt apple=new Äppel();
```

Detta fungerar då äpple ärvt från Fukt, men vi kan inte nå de specifika metoderna och egenskaperna i Äpple, för det måste vi typecasta (eller bara cast) om äpplet till att bli en riktig äpple och inte en " Fukt". Vi anger typen, i detta fall Äppel, inom parentes och tvingar därmed kompilatorn att se frukten som ett äpple.

```
Äppel riktigÄpple=(Äppel) apple;
```

Casting eller typecasting är alltså att byta form på en typ, om typerna är kompatibla på något sätt så kommer de att omvandlas av kompilatorn. Var försiktig, omvandling av fel typ kommer att se bra ut för kompilatorn men kommer att orsaka runtime error.

```
Fukt fr=new Fukt();  
Äppel äpp=(Äppel) fr; //Orsakar runtime error men accepteras av kompilatorn!
```

Använd casting enbart när du är säker på att du kan omvandla från en typ till en annan, annars kan ditt program bli korrupt mitt i körningen.

GARBAGE COLLECTOR

Sophanteraren i .Net. Ett litet program som går igenom alla variabler som lämnats till sitt öde efter att loop, metoder eller classer avslutats. Den letar upp allt som inte längre har någon koppling till programmet och frigör minnet så att Windows minne hålls ren och fin. Man kan forcera fram den vid vanlig exekvering genom att köra kommandot `System.GC.Collect();`

PSEUDOKOD

Pseudo betyder egentligen "falsk" eller "Inte riktig". Pseudokod är inte riktig kod, det är guidelinjer för vad som ska hända i programmet utan att man skriver koden. Oftast använder man det som första steget innan man skapar programmet, för att få en överblick i hur programmet kommer att bli.

Nyckelordet är enkelhet, man skriver enkla kommandon som inte behöver förklaras ytterligare. Det finns tre sorts strukturelement man använder. Sekvens, selektion och repetition. Det låter bekant eller hur? Det finns dock vissa regler, alla strukturelement skrivs med VERSALER, selektioner skrivs med *kursiv stil*. Man kan blanda dessa kodstycken hursomhelst så länge man håller sig till strukturen.

KODEXEMPEL

```
START "Celsius"
FRÅGA grader i Fahrenheit
BERÄKNA Fahrenheit till Celsius
VISA resultat
OM Celsius<10
    SKRIV "Brrrr! Vad kallt!"
ANNARS
    OM Celcius>25
        SKRIV "Varmt och skönt"
SLUT "Celsius"
```

SELEKTION

Selektion är en helt vanlig if-sats, man skriver OM med versaler, selektionen i kursivt och man skriver handlingen som följer indenterat (några steg åt sidan) under OM satsen. Man kan göra flera OM satser i följd efter varandra eller i varandra.

```
OM Celsius<10
    SKRIV "Brrrr! Vad kallt!"
    OM Celsius<-10
        SKRIV "AAGGGHH"
ANNARS
    OM Celcius>25
        SKRIV "Ah, sol, värme!!"
Loop
```

Man kan skriva loopar på olika sätt, vilket också borde vara bekant från C#.

```
SÅLÄNGE Det regnar
    Stanna inne

UPPREPA
    SKRIV "HEJ!"
TILLS det blir tjatigt
```

VAL

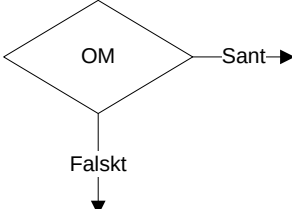
VÄLJ alternativet är helt enkelt en SWITCH sats.

```
VÄLJ fall ur
    månad Januari, Februari: SKRIV "Vinter"
    månad Mars, April, Maj: SKRIV "Vår"
    månad Juni, Juli, Augusti: SKRIV "Sommar"
    månad September, Oktober: SKRIV "Höst"
    månad November, December: SKRIV "Vinter"
```

FLÖDESSCHEMA

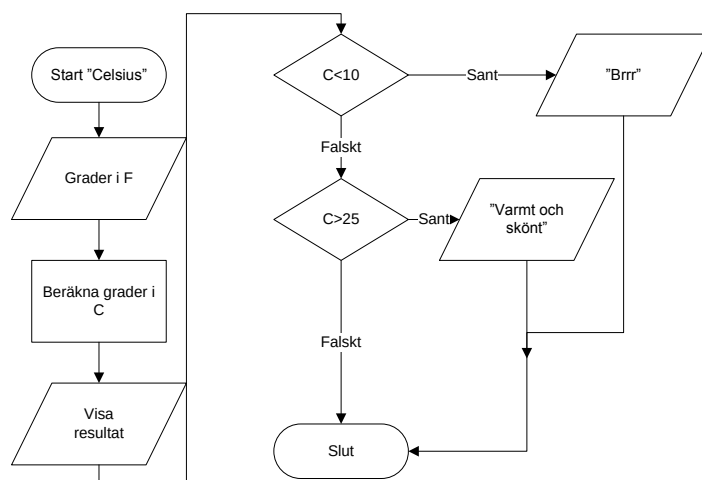
Ett flödesschema är en grafisk form av pseudokod. Man ritat upp koden helt enkelt. Det gör det enklare att överskåda enkla funktioner, men det kan bli extremt kluddigt med stora program.

Precis som pseudokoden så har den en ganska enkel struktur. Man använder olika symboler för att visa olika delar av programmet.

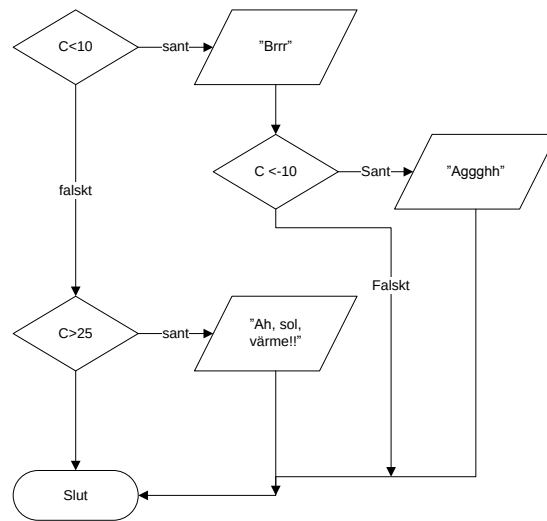
Man börjar alltid med en oval figur, man avslutar med den också. Det finns enbart en början och ett slut på strukturen.	
Sekvenser, beräkningar, databehandling, allt sådant sker i rektanglar	
Input och utput sker i parallelogram	
Villkor anges i en "diamant" eller en liggande romb. Om villkoret är sant går man åt sidan, annars går man neråt.	
Man kopplar ihop bilderna med pilar, ibland är det bra att skriva på pilen vad som kommer att ske efteråt, eller varför man väljer den vägen.	

Nedan följer några exempel på flödesscheman, dessa gör exakt samma sak som pseudokoden i tidigare exempel.

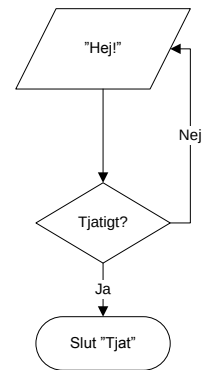
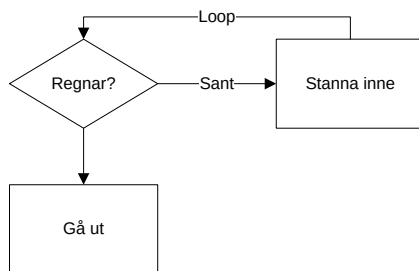
KODEXEMPEL



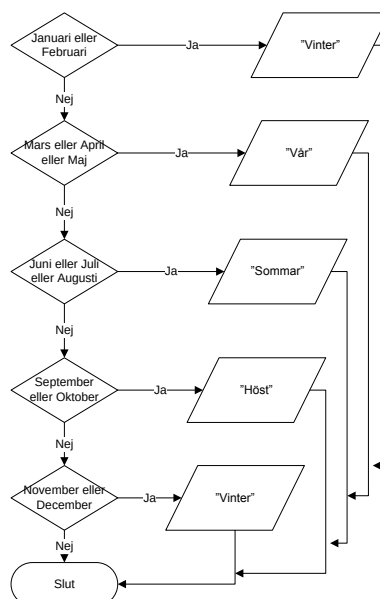
SELEKTION



LOOP



VAL



KODEXEMPEL

Här nedan följer exempel på kodsnuttar som förklarar olika programmeringssituationer, dessa är inte fullständiga program så ibland måste de läggas in i main() metoden, och ibland läggs de in som classer.

CONSTRUCTOR

```
class MinClass
{
    //Constructor som inte tar emot något
    public MinClass()
    { }

    //Constructor som tar emot en Int
    public MinClass(int X)
    { }

    //Constructor som tar emot en sträng och en int
    public MinClass(String s, int i)
    { }
}
```

DESTRUCTOR

```
class MinClass
{
    ~MinClass()
    { Console.WriteLine("Nu är min class död"); }
}
```

DESTRUCTOR TRIGGER

```
MinClass MC = new MinClass();
MC = null;
//Anropas för att köra igång Garbage collectorn
//annars kommer Destructor att triggas när main() avslutas
System.GC.Collect();
Response.Read();
```

INSTANCIERING

```
//Instanciering utan parametrar
MinClass MC = new MinClass();

//Instanciering med en string som parameter
MinClass MC = new MinClass(MinString);

//Instanciering med en string som parameter
MinClass MC2 = new MinClass("Hej");

// Instanciering med en char och int som parameter
MinClass MC = new MinClass('G', 15);
```

CLASSMETOD

```
class MinClass
{
    public static void MinMetod(int X)
    {}
}
```

VARIABLER

```
//Deklaration av variabler
int eger;
double Agent;
string orchestra;
MinClass ClassInstans;

//Deklaration av variabler
Tilldelning av värden
eger=5;
Agent=.007;
orchestra="classic";
ClassInstans = new MinClass();
eger++;
```

STATISK FÄLT

Detta kodexempel visar hur static värdet överlever instanciering och förstörelse av en class.

```
class StatiskFält
{
    public static int Nummer=0;

    public void Plus()
    {
        Nummer++;
    }
}

class Program
{
    static void Main(string[] args)
    {
        StatiskFält.Nummer = 8; //Sätter statisk värde
        StatiskFält Test = new StatiskFält();
        StatiskFält NyTest = new StatiskFält();
        Test.Plus(); //Adderar 1 till det statiska värdet
        NyTest.Plus(); //Adderar 1 till det statiska värdet
        Test = null; //Förstör Test
        NyTest = null; //Förstör NyTest
        Console.WriteLine(StatiskFält.Nummer);
        Console.ReadLine(); //Skriver ut 10
    }
}
```

METODER

```
//Public metod som returnerar en int
public int HämtaEttVärde() { return X;}

//Public metod som tar emot en int och returnerar ingenting
public void SkickaInEttVärde(int X) { }
```

OVERLOAD

Overload är när man gör flera versioner av en metod, fast med olika parametrar. Vanligen gör man det med **constructorn** i en **class** när man har flera stycken i samma class, men det fungerar fint med vanliga metoder också. Man får inte ha overrides som tar emot exakt samma sorts parametrar i exakt samma ordning.

Exempelvis:

```
public void SkrivUt() { /*Gör något här */ }
public void SkrivUt(int x) { /*Gör något här */ }
public void SkrivUt(string s) { /*Gör något här */ }

// Ej tillåtet för att det finns en sådan redan,
// även om variabelnamnet skiljer sig
public void SkrivUt(string x) { /*Gör något här */ }

//Detta fungerar, men rekommenderas inte
public void SkrivUt(string s, int x) { /*Gör något här */ }
public void SkrivUt(int x, string s) { /*Gör något här */ }
```

METODANROP

```
//Anropar metoden Fråga, som skriver ut texten, frågar om ett tal
//omvandlar det till en int och returnerar värdet, den testas även
//så att man matat in siffror, annars returneras noll.
//Allt det utförs i en enda rad!!!
int antalDörrar = Fråga("Hur många dörrar har bilen (inkl baklucka)");

private static int Fråga(string minFråga)
{
    Console.WriteLine("{0} ",minFråga);
    int Svar=0;
    string s=Console.ReadLine();
    int.TryParse(s, out Svar);
    return Svar;
}
```

VARIABELNAMN

```
//Anständiga variabelnamn enligt Ungersk notation
int iAntalDörrar;
double dHastighetPerSekund;
string sFörnamn;

//Anständiga variabelnamn enligt Pascal Case
int antalDörrar;
double hastighetPerSekund;
string förnamn;

//Korkade variabelnamn
int sAntalDörrarPåMinOtroligtSnyggBil;
double HerreJissesVadDetgårSnabbtMeddenhärBilen;
string HärHarViNamnetPåPersonenSomKörProgrammetJustNu;

//Intetsägande variabelnamn
int a;
double b;
string c;
```

INSTANSMETODER OCH INSTANSVARIABLER

```
//Instanciera klassen
MinClass MC = new MinClass();

//Tilldela instansvariabeln ett värde
MC.MittInt = 29;

//Anropa instansmetoden
MC.MinMetod();

class MinClass
{
    public int MittInt; //Instansvariabel
    public void MinMetod() //InstansMetod
    { }
}
```

CONST

```
//-----  
//Exempel på hur man kan använda konstanter i kodningen  
//-----  
  
const int antalMånader = 12; //Antal månader på ett år  
  
const int a4Marginal = 3; //Antal cm i marginalen  
  
const string appNamn = "Mitt program"; //Namnet på ditt program  
const string programmerare = "Ditt namn"; //Programmerarens namn  
  
const int maxBredd = 80; //Max bredd för ett consolfönster  
const int maxHöjd = 25; //Max höjd för ett consolfönster
```

CONST I PRAKTIKEN, EXEMPEL 1

```
const int December=12;  
const int Juldagen=25;  
  
int månad = DateTime.Now.Month; //Aktuell månad  
int dag = DateTime.Now.Day; //Aktuell dag  
  
//Läsbar kod där man förstår snabbt vad den gör  
if (månad==December && dag<Juldagen)  
    Console.WriteLine("Snart är det jul");
```

CONST I PRAKTIKEN, EXEMPEL 2

```
const int maxPos = 5; //Maxvärde i X led  
const int minPos = 1; //Minvärde i X led  
const int antalSteg = 1; //Steg per iteration  
  
int posRiktning = antalSteg; //Riktning för räknandet  
int pos = 0; //Startposition  
  
//Loop för att se hur räknaren fungerar  
for (int Räkna = 0; Räkna < 22; Räkna++)  
{  
    //Öka positionsräknaren 1 steg i aktuell riktning  
    pos += posRiktning;  
  
    //Kontrollera om den är utanför gränserna, ändra riktning isåfall  
    if (pos >= maxPos)  
        posRiktning = -antalSteg; //Gå i negativ riktning  
    if (pos <= minPos)  
        posRiktning = antalSteg; //Gå i positiv riktning  
  
    //Skapa utdatan  
    //String.Concat slår ihop flera strängar  
    string utdata = string.Concat(pos.ToString(), " ",  
                                   new string(' ', pos), "*");  
  
    Console.WriteLine(utdata);  
}
```

REF

```
string Namn="Medina"; //Måste initieras
SnyggaUppNamn(ref Namn);
Console.WriteLine(Namn);

private void SnyggaUppNamn(ref string Namn)
{
    Namn = "Mr " + Namn;
}
```

OUT

```
string Namn;
AngeNamn(out Namn);
Console.WriteLine(Namn);

private void AngeNamn(out string Namn)
{
    Namn = "Mr Medina";
}
```

BYVAL OCH BYREF

```
class MinClass
{
    public int MittVärde { get; set; }
    public MinClass(int värde) { this.MittVärde = värde; }
}

//int är ByVal så x kommer aldrig att ändras i metoden som anropade
private static void ÄndraVärdet(int x) { x += 5; }
//string är ByVal så s kommer aldrig att ändras i metoden som anropade
private static void ÄndraVärdet(string s) { s += "!!!"; }
//Classer är ByRef så värdet i dem kan ändras i metoden
private static void ÄndraVärdet(MinClass mc) { mc.MittVärde += 10; }

static void Main(string[] args)
{
    int x = 10;

    ÄndraVärdet(x);
    Console.WriteLine(x);

    string s = "Hej";
    ÄndraVärdet(s);
    Console.WriteLine(s);

    MinClass mc = new MinClass(5);
    ÄndraVärdet(mc);
    Console.WriteLine(mc.MittVärde);
}
```

```
string s="";
int i=0;
double d=0;

Console.WriteLine(s.GetType());
Console.WriteLine(i.GetType());
Console.WriteLine(d.GetType());

//Resultat:
//System.String
//System.Int32
//System.Double
//Jämförelse med GetType
```

```
private void KollaTyp(object s)
{
    //Jämförelse med GetType
    if (s.GetType() == typeof(string))
        Console.WriteLine("s är en sträng");
}
```

```
private void KollaTyp(object s)
{
    //Jämförelse med GetType
    if (s is string)
        Console.WriteLine("s är en sträng");
}
```

```
//Förbered olika variabler i minnet
int x = 5;
double d = 2.5;
string a = "a";
string b = "b";
string ab = "ab";

//Skriv ut hashvärdena
Console.WriteLine("Typ\t\t\t\t\t Värde\t\t\t\t Hashcode");
Console.WriteLine(new string('-', 50));
Console.WriteLine("int\t\t\t\t\t : {0}\t\t\t --> \t{1}", x, x.GetHashCode());
Console.WriteLine("double\t\t\t\t\t : {0}\t\t\t --> \t{1}", d, d.GetHashCode());
Console.WriteLine("string\t\t\t\t\t : {0}\t\t\t --> \t{1}", a, a.GetHashCode());
Console.WriteLine("string\t\t\t\t\t : {0}\t\t\t --> \t{1}", b, b.GetHashCode());
Console.WriteLine("string\t\t\t\t\t : {0}\t\t\t --> \t{1}", ab,
ab.GetHashCode());
```

```
//Resultatet blir detta
//Typ          Värde          Hashcode
//-----
//int           : 5             -->    5
//double        : 2,5          --> 1074003968
//string        : a             --> -842352705
//string        : b             --> -842352706
//string        : ab            --> -840386625
```

HASHVÄRDEN OCH CLASSER

```
NamnClass mc = new NamnClass();
NamnClass mc2 = new NamnClass();
NamnClassPlus mnc = new NamnClassPlus();
NamnClassPlus mnc2 = new NamnClassPlus();

mc.Namn = "Jake Sully";
mc2.Namn = "Jake Sully";

mnc.Namn = "Neytiri";
mnc2.Namn = "Neytiri";

//Skriv ut hashvärdena
Console.WriteLine("Typ          Värde\t\t\t Hashcode");
Console.WriteLine(new string('-',50));
Console.WriteLine("NamnClass      : {0}\t --> \t{1}", mc.ToString(),
mc.GetHashCode());
Console.WriteLine("NamnClass      : {0}\t --> \t{1}", mc2.ToString(),
mc2.GetHashCode());
Console.WriteLine();
Console.WriteLine("NamnClassPlus : {0}\t\t --> \t{1}", mnc.ToString(),
mnc.GetHashCode());
Console.WriteLine("NamnClassPlus : {0}\t\t --> \t{1}", mnc2.ToString(),
mnc2.GetHashCode());
Console.Read();

//Resultatet blir detta
//Typ          Värde          Hashcode
//-----
//NamnClass     : Jake Sully    -->    45653674
//NamnClass     : Jake Sully    -->    41149443
//
//NamnClassPlus : Neytiri       -->    1067895399
//NamnClassPlus : Neytiri       -->    1067895399

//NamnClassPlus ger samma Hashvärde om egenskapen namn är likadan
//det gör inte NamnClass!
```

HASHCODE

```
class Person
{
    public string Namn { get; set; }
    public string Personnr { get; set; }

    //Överridar GetHashCode metoden för att ge tillbaka
    //personnumrets hashcode, istället för classens
    public override int GetHashCode()
    {
        return this.Personnr.GetHashCode();
    }

    //Överridar Equals metoden
    //Jämför två Personobjekt mot varandra
    //baserat på personnummer
    public override bool Equals(object obj)
    {
        if (obj == null)
            return false;

        //Kontrollera att objektet som skickades in
        //är en person
        if (obj is Person)
        {
            Person Jämför = (Person)obj;
            return this.Personnr == Jämför.Personnr;
        }
        else
        {
            return false;
        }
    }
}

//HashSet är en array utan dubletter
HashSet<Person> Personreg = new HashSet<Person>();
Person p = new Person();

//Testperson 1
p.Namn = "Marcus";
p.Personnr = "851012-4455";
Personreg.Add(p);

//Testperson 2, har samma personnummer som Testperson 1
p = new Person();
p.Namn = "Johan";
p.Personnr = "851012-4455";
Personreg.Add(p);

foreach (Person person in Personreg)
{
    Console.WriteLine("{0} {1} {2}",
        person.Namn, person.Personnr, person.GetHashCode());
}
//Enbart Testperson 1 skrivs ut
```

ALGORITM

Här är ett exempel på en serie instruktioner för hur du ska räkna ut ett tal.

- 1) Tänk på ett tal
- 2) Multiplicera det med 2
- 3) Lägg till 10
- 4) Dela det med 2
- 5) Minska det med talet du tänkte på

Du har nu talet 5

I C# skulle det se ut såhär:

```
Random rnd = new Random();  
int x = rnd.Next(20);  
int y = x * 2;  
y += 10;  
y /= 2;  
y -= x;  
Console.WriteLine(y);
```

HUND-CLASS

```
//Bas hundclass med standardegenskaper för en hund  
class Hund  
{  
    public int antalBen = 4;  
    public string färg = "Brun";  
}  
//Chihuahua class som ärver bashunden  
class Chihuahua : Hund  
{  
    public bool bjäbbig = true;  
}  
//Pitbull class som ärver bashunden  
class Pitbull : Hund  
{  
    public bool gosig = true;  
}  
//Pudel class ärver från chihuahua  
class Pudel : Chihuahua {}  
  
Chihuahua tacoBell = new Chihuahua();  
  
//Man kan ange färg på hunden även om det inte deklarerades  
//i Chihuahua classen  
tacoBell.färg = "Blågrå";  
tacoBell.bjäbbig = false;  
  
//Pudeln har ingenting deklarerat i sin class, men den har alla  
//egenskaper som Chihuahuan har.  
Pudel vov = new Pudel();  
vov.bjäbbig = true;  
vov.färg = "vit";
```

POLYMORFISM

//Vi lägger till en metod i programclassen

```
private static void VisaHundInfo(Hund Jycke)
{
    Console.WriteLine(
        "Hunden är av {0} färg och har {1} ben",
        Jycke.färg,
        Jycke.antalBen);
}
```

//Och fyller på koden från förra exemplet med detta

```
VisaHundInfo(vov); //Vov är en pudel
VisaHundInfo(tacoBell); //TacoBell är en Chihuahua
```

```
//Resultat:
//Hunden är av vit färg och har 4 ben
//Hunden är av Blågrå färg och har 4 ben
```

BASE OCH THIS

```
class SuperClass
{
    public virtual void Hej() {Console.WriteLine("Hej!");}
}
class SubClass:SuperClass
{
    string meddelande = "Goddag!";

    public override void Hej()
    {
        //Anropar superclassens metod först
        base.Hej();
        Console.WriteLine("....hejdå");
    }
    public void VisaMeddelande() {
        //Läser av variabel inom den egna classen
        Console.WriteLine(this.meddelande);
    }
}
```

SEALED

```
class SuperClass
{
    public virtual void Hej()
    { Console.WriteLine("Hello"); }
}

class SubClass1 : SuperClass
{
    public override sealed void Hej()
    { Console.WriteLine("Hello World"); }
}

class SubClass2 : SubClass1
{
    //Detta kommer inte att accepteras av kompilatorn
    //public override void Hej()
    //{ Console.WriteLine("Hello Universe!!"); }

    //Detta fungerar däremot, men är inte kompatibel med polimorfism
    public new void Hej() { Console.WriteLine("Hello Universe!!"); }
}
```

ABSTRAKT CLASS

```
//Class för att definiera en standard pepparkaksgubbe
abstract class Pepparkaksgubbe
{
    public Arm Armarna { get; set; }
    public Ben Benen { get; set; }
    public Huvud Huvudet { get; set; }
    public Kropp Kroppen { get; set; }
}
```

INTERFACE

```
//Interface som alla andra hänvisar till
interface Kroppsdel { /* Kod för kroppsdel */ }

interface DubblaKroppsdelar // Detta interface ärvs av andra
{
    Kroppsdel Höger { get; set; }
    Kroppsdel Vänster { get; set; }
}

//Armarnas längd och data för höger och vänster arm
interface Arm :DubblaKroppsdelar
{
    int Längd { get; set; }
}

//Benens längd och data för höger och vänster ben
interface Ben:DubblaKroppsdelar
{
    int Längd { get; set; }
}

//Huvudets form
interface Huvud
```

```

{
    Kroppsdel Huvudet { get; set; }
}
//Kroppens form
interface Kropp
{
    Kroppsdel Torso{ get; set; }
}
//Ögonens färg och data för höger och vänster öga
interface Ögon:DubblaKroppsdelar
{
    string Färg{ get; set; }
}

```

ICOMPARABLE

```

class MinClass : IComparable
{
    public int MittVärde { get; set; }
    public int CompareTo(object obj)
    {
        //Kolla om objektet är null
        if (obj == null)
            return 1;
        //Kolla om objektet är av rätt class
        if (obj is MinClass)
        {
            //Typecasta objektet
            MinClass Jämför = (MinClass)obj;

            //Jämför värdena
            if (this.MittVärde < Jämför.MittVärde)
                return -1; //Mitt värde är mindre
            else if (this.MittVärde > Jämför.MittVärde)
                return 1; //Mitt värde är större
            else
                return 0; //Båda värdena är lika
        }
        else
        {
            return 1;
        }
    }
}

```

ICOMPARABLE – ENKLARE VERSION

```
class MinClass : IComparable
{
    public int MittVärde { get; set; }
    public int CompareTo(object obj)
    {
        //Kolla om objektet är null
        if (obj == null)
            return 1;
        //Kolla om objektet är av rätt class
        if (obj is MinClass)
        {
            //Typecasta objektet
            MinClass Jämför = (MinClass)obj;

            //Detta är möjligt för att "MittVärde" är en int
            //och int har CompareTo implementerat
            return this.MittVärde.CompareTo(Jämför.MittVärde);
        }
        else
        {
            return 1;
        }
    }
}
```

ICOMPARABLE <T>

```
class MinClass : IComparable<MinClass>
{
    public int MittVärde { get; set; }
    public int CompareTo(MinClass obj)
    {
        //Kolla om objektet är null
        if (obj == null)
            return 1;

        //Jämför värdena
        if (this.MittVärde < obj.MittVärde)
            return -1; //Mitt värde är mindre
        else if (this.MittVärde > obj.MittVärde)
            return 1; //Mitt värde är större
        else
            return 0; //Båda värdena är lika
    }
}
```

ICOMPARABLE<T> – ENKLARE VERSION

```
class MinClass : IComparable
{
    public int MittVärde { get; set; }
    public int CompareTo(MinClass obj)
    {
        //Kolla om objektet är null
        if (obj == null)
            return 1;

        //Detta är möjligt för att "MittVärde" är en int
        //och int har CompareTo implementerat
        return this.MittVärde.CompareTo(obj.MittVärde);
    }
}
```

IENUMERATOR

```
//Returnerar det aktuella objektet i arrayen
public object Current
{
    get { return GrundClass.dataArray[index]; }
}

//MoveNext ökar indexet med ett och returnerar
//true om man kan fortsätta iterera listan eller arrayen
//false om man kommit till slutet av listan eller arrayen
public bool MoveNext()
{
    index++;
    return (index < GrundClass.dataArray.Length);
}

//Reset nollställer indexet så att listan visas om från
//början.
public void Reset()
{
    index = 0;
}
```

GENERISK CLASS

```
//En class som kan fungera som olika typer
class MinGeneriskaClass<T>
{
    public T MittVärde { get; set; }
    public MinGeneriskaClass(T värde) {this.MittVärde = värde;}
    public override string ToString() {return MittVärde.ToString();}
}

MinGeneriskaClass<int> i = new MinGeneriskaClass<int>(42);
MinGeneriskaClass<string> s = new MinGeneriskaClass<string>("Hello World");

Console.WriteLine(i);
```

```
Console.WriteLine(s);
Console.Read();
```

GET/SET METODER

```
Glasskiosk gk = new Glasskiosk();
gk.SetPucko(10);
Console.WriteLine("Det finns {0} puckon på lager",gk.GetPucko());

//Här använder vi två metoder för att läsa av och sätta ett värde
public class Glasskiosk
{
    private int pucko = 0;

    public int GetPucko()
    {
        return pucko;
    }

    public void SetPucko(int tal)
    {
        pucko = tal;
    }
}
```

EGENSKAPER

```
Glasskiosk gk = new Glasskiosk();
gk.X15    = 10;
gk.Pucko  = 11;

Console.WriteLine("Så många Puckos finns det: {0}",gk.Pucko);
Console.WriteLine("Det finns {0} X15",gk.X15);

public class Glasskiosk
{
    private int pucko = 0;
    private int x15 = 0;

    // Här kan man sätta och läsa av värdet på X15 fältet
    public int X15
    {
        get { return x15; }
        set { x15 = value; }
    }

    // Här kan man sätta och läsa av värdet på Pucko fältet
    public int Pucko
    {
        get { return pucko; }
        set { pucko = value; }
    }
}
```

EN ANNORLUNDA EGENSKAP

```
Person Marc = new Person();
Marc.Födelsedag = new DateTime(1989, 12, 13);
Console.WriteLine(Marc.Ålder);

public class Person
{
    DateTime födelsedag = DateTime.Now;

    //DateTime är ett objekt som hjälper en med datumhantering
    public DateTime Födelsedag
    {
        get { return födelsedag; }
        set { födelsedag = value; }
    }

    public int Ålder
    {
        //Denna egenskapen lämnar ifrån sig åldern på personen, även
        //om det inte finns ett fält som håller reda på åldern i sig.
        //Åldern beräknas baserat på fältet som anger personens födelsedag.
        //Denna egenskapen är "skrivskyddad", den kan inte skrivas till
        //(det finns ingen set metod)
        get
        {
            DateTime iÅr = new DateTime(DateTime.Now.Year,
                                         födelsedag.Month, födelsedag.Day);
            TimeSpan PasseradTid = iÅr - födelsedag;
            return (int)(PasseradTid.Days / 365.25);
        }
    }
}
```

LOOP-EXEMPEL

OÄNDLIGA LOOPAR

```
//Detta kan orsaka en oändlig loop om värde1 inte ändras i loopen
//Eller om värdet ökar, exempelvis från 9 till 11
//det är bättre att använda större än/mindre än jämförelser
while (värde1 != 10) { //utför diverse beräkningar }

//Detta ser rätt ut, men break kommer aldrig att triggas
while (true) {
    värde1++;
    if (värde1 % 2 == 0) värde1++;
    if (värde1 == 16) break;
}

//Det är farligt att leka okontrollerat med
//räknaren i en for-loop.
for (int räknare = 5; räknare > 1; räknare--) {räknare++;}

//En felskriven forsats kan ställa till med
//mycket gråt och tandagnisslan
for (int räknare = 5; räknare > 1; räknare++)
{ /*utför diverse beräkningar*/ }
```

```
//Om ingen if-sats med break stoppas in i koden kommer
//detta att köra för evigt
while (true)
{ /*utför diverse beräkningar*/ }
```

FOR-SATS

```
//Skriv ut bokstäver
for (int räknare = 1; räknare < 27; räknare++)
{
    Console.Write("{0}", (char)(räknare+96));
}
Console.WriteLine();
```

WHILE-SATS

```
//Försöker hitta ett schysst pris på en dator
//Här vet man inte hur många gånger man ska loopa
double datorPris=12500;
while (datorPris > 8000)
{
    datorPris*=.98;
}
Console.WriteLine("Billigaste datorn går på:{0}",Math.Round(datorPris,2));
```

DO-SATS

```
//Kollar om personen är myndig
//Man vet inte antalet frågor som ska ställas, det kan vara en
//och det kan bli 100, beroende på svaren som datorn får.
int ålder = 25;
do
{
    Console.Write("Hur gammal är du? ");
    ålder = int.Parse(Console.ReadLine());
} while (ålder < 18);
Console.WriteLine("Ok, du är myndig.");
```

NÄSTLADE LOOPAR

```
// Nästlad while-loop som genererar
// en mysko bild <--->
int x = 0;
int y = 0;
while (x++ < 5)
{
    Console.Write("<");
    y = 0;
    while (y++ < x)
    {
        Console.Write("-");
    }
    Console.WriteLine(">");
}
```

LUFFARSCHACK

```
//Skapa en karta för luffarschack
//OBS! Detta är inte ett färdigt spel
//Den får du skriva själv :)
int[,] luffarShack = new int[3, 3];
for (int posY = 0; posY < 3; posY++)
{
    for (int posX = 0; posX < 3; posX++)
    {
        switch (luffarShack[posY, posX])
        {
            case 1: Console.Write("O") ; break;
            case 2: Console.Write("X") ; break;
            default: Console.Write(" "); break;
        }
        if (posX < 2) Console.Write("|");
    }
    Console.WriteLine();
    if (posY<2) Console.WriteLine("-----");
}
```

KLOCKA

```
//Klocka som kommer att köra tills midnatt
for (int timmar = DateTime.Now.Hour; timmar < 24; timmar++)
{
    for (int minuter = DateTime.Now.Minute; minuter < 60; minuter++)
    {
        for (int sekunder = DateTime.Now.Second; sekunder < 60; sekunder++)
        {
            Console.SetCursorPosition(10, 10);
            Console.WriteLine("Klockan är {0:00}:{1:00}:{2:00}",
                              timmar, minuter, sekunder);
            System.Threading.Thread.Sleep(1000);
        }
    }
}
```

ARRAYER

```
//Endimensionell array
string[] klasslista=new string[52];

//Tvådimensionell array
int[,] luffarSchack = new int[3, 3];

//Tredimensionell array
KubSida[, ,] rubiksKub = new KubSida[3, 3, 3];
```

HANDELLISTA

Här är ett praktiskt exempel på hur man använder arrays.

```
int antalVaror = 3;
Varor[] handelLista=new Varor[antalVaror];
HandelLista[0] = new Varor();
HandelLista[1] = new Varor();
HandelLista[2] = new Varor();
handelLista[0].Namn = "Tröja";
handelLista[0].Pris = 107;
handelLista[1].Namn = "Dataspel";
handelLista[1].Pris = 329;
handelLista[2].Namn = "Bok";
handelLista[2].Pris = 150;
double summa=0;
for (int addera = 0; addera < handelLista.Length; addera++)
    summa+= handelLista[addera].Pris;
Console.WriteLine("Du får lägga ut {0}",summa);

class Varor
{
    private string namn;
    private double pris;

    public string Namn
    { get { return namn; }
      set { namn = value; }}

    public double Pris
    {get { return pris; }
     set { pris = value; }}
}
```

ARRAYPOSITIONER

```
string[] namnLista =
{
    "Jake Sully",
    "Dr. Grace Augustine",
    "Neytiri",
    "Colonel Miles Quaritch",
    "Trudy Chacon"
};

int förstaNamn = 0; //Första är alltid 0
int mittenNamn = namnLista.Length / 2;
int sistaNamn = namnLista.Length - 1;
int antalNamn = namnLista.Length;

Console.WriteLine("Första :{0} {1}", förstaNamn, namnLista[förstaNamn]);
Console.WriteLine("Mitten :{0} {1}", mittenNamn, namnLista[mittenNamn]);
Console.WriteLine("Sista :{0} {1}", sistaNamn, namnLista[sistaNamn]);
Console.WriteLine("Antal namn i listan:{0}", antalNamn);

//Resultatet blir:
//Första :0 Jake Sully
//Mitten :2 Neytiri
//Sista :4 Trudy Chacon
//Antal namn i listan:5
```

INTERNET LÄNKAR

Objektorienterad programmering:

http://www.nada.kth.se/kurser/kth/2D1311/02-03/V/foerelaesningar/utskriftsversion/Objektorienterad_programmering.pdf

Webbstudio InternetWorld

<http://iwtjanster.idg.se/webbstudio/pub/avdelning.asp?id=23>

C-Sharp Skolan

<http://www.csharpskolan.se/>

MSN-videos med C# lektioner

<http://msdn.microsoft.com/en-us/vcsharp/aa336766.aspx>

C Sharp Tutorial

http://www.meshplex.org/wiki/C_Sharp_Tutorial

Grafik till programmen

<http://www.iconarchive.com/>

<http://www.freeiconsdownload.com/>

<http://www.freeiconsweb.com/>

Nerdkultur

<http://xkcd.com/>